

浙江大学

本科实验报告

课程名称：计算机组成与设计

姓名：张序鸿

学院：竺可桢学院

专业：计算机科学与技术

学号：3230103265

指导教师：杨莹春

2024 年 11 月 15 日

浙江大学实验报告

课程名称：____ 计算机组成与设计 ____ 实验类型：____ 综合 ____

实验项目名称：____ IP 核集成设计 CPU ____

学生姓名：____ 张序鸿 ____ 专业：____ 混合班 ____ 学号：____ 3230103265 ____

指导老师：____ 杨莹春、潘之杰 ____ 助教：____ 张立冬、马致远 ____

实验地点：____ 东 4-509 ____ 实验日期：____ 2024 年 11 月 20 日 ____

一、实验目的和要求

1. 复习寄存器传输控制技术
2. 掌握 CPU 的核心组成：数据通路与控制器
3. 设计数据通路的功能部件
4. 进一步了解计算机系统的基本结构
5. 熟练掌握 IP 核的使用方法

二、实验内容和原理

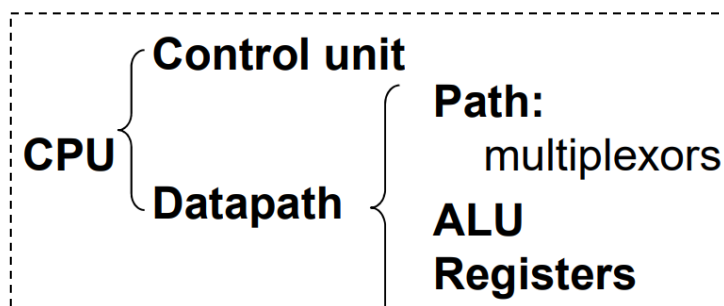
内容：

任务一：利用数据通路和控制器两个 IP 核集成设计 CPU

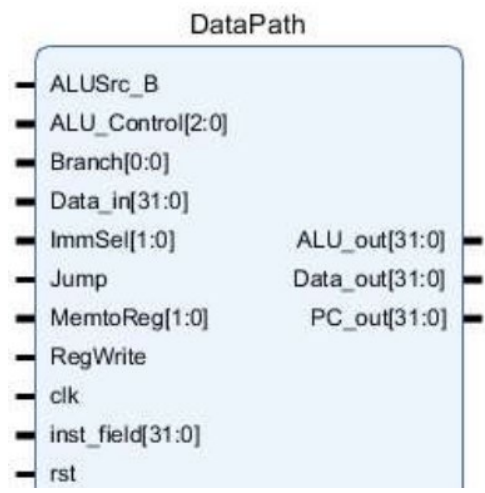
—— 根据原理图采用 RTL 代码实现

原理：

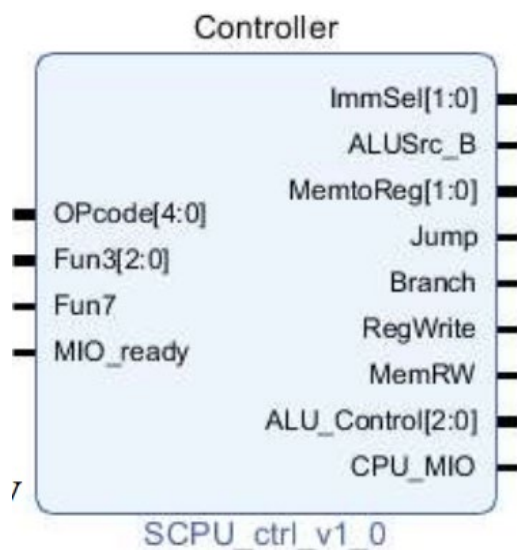
2.1 计算机系统中 CPU 组成



• 数据通路 Data_path 作为 CPU 主要部件之一，具有通用计算功能的算术逻辑部件（主要为 ALU）、具有通用目的寄存器堆、具有通用计数所需的尽可能的路径。



• 控制器 SCPU_ctrl 是寄存器传输控制技术中的运算和通路控制器，它起到指令译码（inst_in 的分块）、产生操作控制信号（ALU 运算控制）、产生指令所需的路径选择的作用。



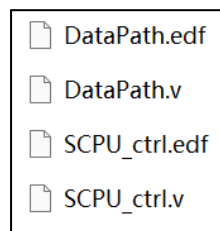
三、实验过程和数据记录

3.1 RTL 集成设计实现 CPU:

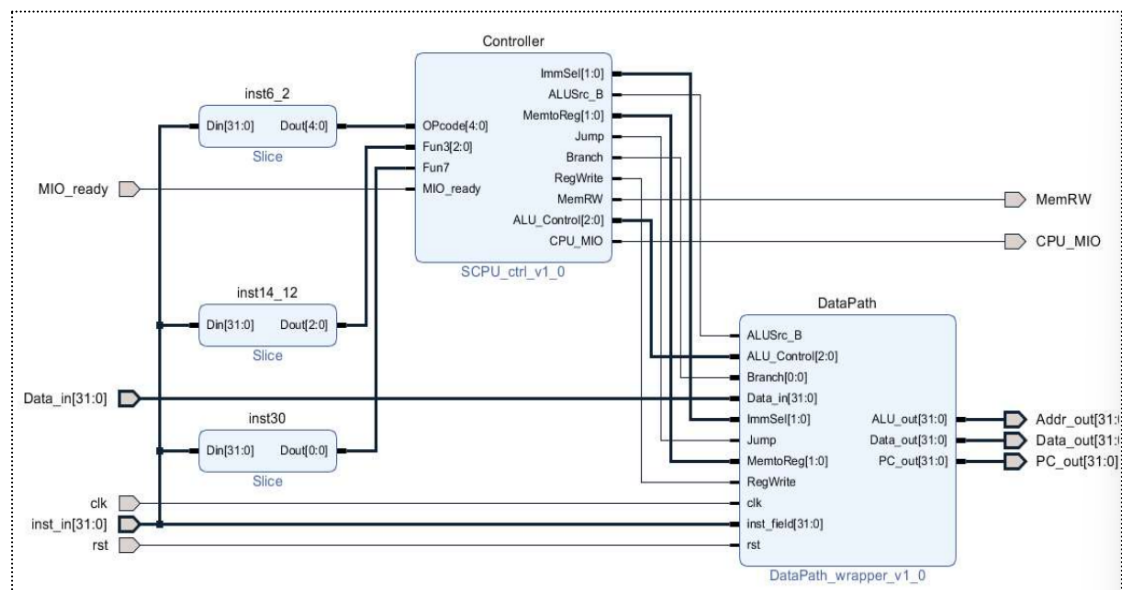
(1) Vivado 新建工程

- 利用 Vivado 新建工程，命名 OExp04-IP2CPU ；
- 最后正确选择 FPGA 芯片——xc7a100tcsq324-1；

本次实验未对 Datapath、SCPU_ctrl 进行具体逻辑实现，采用调用 2 个封装 IP 的方式完成 CPU 的基础模块调用和连接。



(2) RTL 代码描述设计 CPU:



逻辑原理图如上，代码实现如下：

```
`timescale 1ns / 1ps

module SCPU(
    input clk,
    input rst,
    input [31:0]Data_in,
    input [31:0]inst_in,
    input MIO_ready,
```

```

output MemRW,
output CPU_MIO,
output [31:0]Data_out,
output [31:0]PC_out,
output [31:0]Addr_out
);

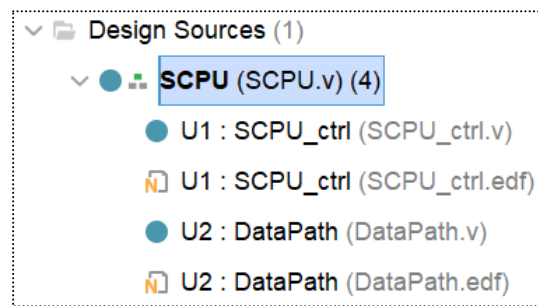
wire [2:0]ALU_Control;
wire [1:0]ImmSel;
wire [1:0]MemtoReg;
wire Jump, Branch, RegWrite, ALUSrc_B;

SCPU_ctrl U1(
    .OPcode(inst_in[6:2]),
    .Fun3(inst_in[14:12]),
    .Fun7(inst_in[30]),
    .MIO_ready(MIO_ready),
    .ImmSel(ImmSel),
    .ALUSrc_B(ALUSrc_B),
    .MemtoReg(MemtoReg),
    .Jump(Jump),
    .Branch(Branch),
    .RegWrite(RegWrite),
    .ALU_Control(ALU_Control),
    .CPU_MIO(CPU_MIO)
);

DataPath U2(
    .ALUSrc_B(ALUSrc_B),
    .ALU_Control(ALU_Control),
    .Branch(Branch),
    .Data_in(Data_in),
    .ImmSel(ImmSel),
    .Jump(Jump),
    .MemtoReg(MemtoReg),
    .RegWrite(RegWrite),
    .clk(clk),
    .inst_field(inst_in),
    .rst(rst),
    .ALU_out(Addr_out),
    .Data_out(Data_out),
    .PC_out(PC_out)
);
endmodule

```

(3) Lab04-0 工程结构：



目前,SCPU 部分仅引用网表文件实现,后续实验旨在逻辑实现 SCPU_ctrl 和 Datapath 模块,因此本实验的连线部分不能出错。

四、实验结果分析

本实验知识 Lab4 的基础,仅实现了 CPU 对 Datapath、SCPU_ctrl 的调用和连接,因此没有实验的相关结果分析。

五、讨论与心得

本实验将 Lab2 中的 SCPU.edf 进行了分解,主要目的还是让我们知道 SCPU 的组成部分,主要抽象成 Datapath、SCPU_ctrl 两部分。

后续实验就是在此实验基础上,替换 Datapath.edf、SCPU_ctrl.edf 两个文件,利用逻辑实现我们所学习的 CPU,并完成一定的指令拓展和中断操作,所以,“好日子”还在后头。

浙江大学

本科实验报告

课程名称：计算机组成与设计

姓名：张序鸿

学院：竺可桢学院

专业：计算机科学与技术

学号：3230103265

指导教师：杨莹春

2024 年 11 月 15 日

浙江大学实验报告

课程名称: 计算机组成与设计 实验类型: 综合

实验项目名称: CPU 设计之数据通路

学生姓名: 张序鸿 专业: 混合班 学号: 3230103265

指导老师: 杨莹春、潘之杰 助教: 张立冬、马致远

实验地点: 东 4-509 实验日期: 2024 年 11 月 20 日

一、实验目的和要求

1. 复习寄存器传输控制技术
2. 掌握 CPU 的核心组成: 数据通路与控制器
3. 掌握 CPU 的核心: 数据通路组成与原理, 并设计 Datapath
4. 学习测试方案的设计

二、实验内容和原理

内容:

任务一: 设计实现数据通路 —— 采用 RTL 实现

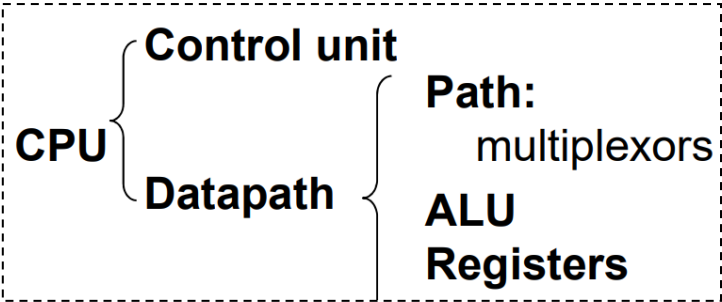
- ALU 和 Regs 调用 Exp01 设计的模块
- PC 寄存器设计及 PC 通路建立 π
- ImmGen 立即数生成模块设计
- 此实验在 Exp4-0 的基础上完成, 替换 Exp4-0 的数据通路核

任务二: 设计数据通路测试方案并完成测试

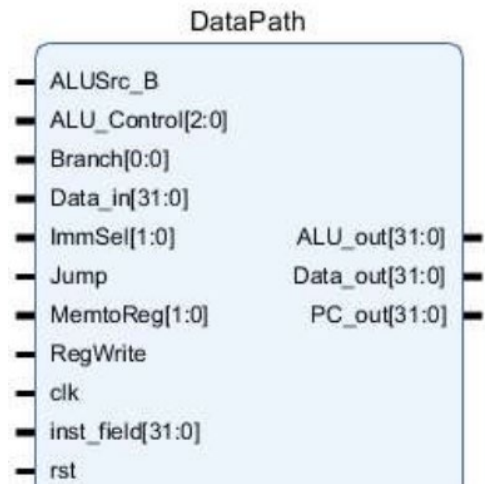
- 通路测试: I-格式通路、R-格式通路
- 部件测试: ALU、Register Files

原理：

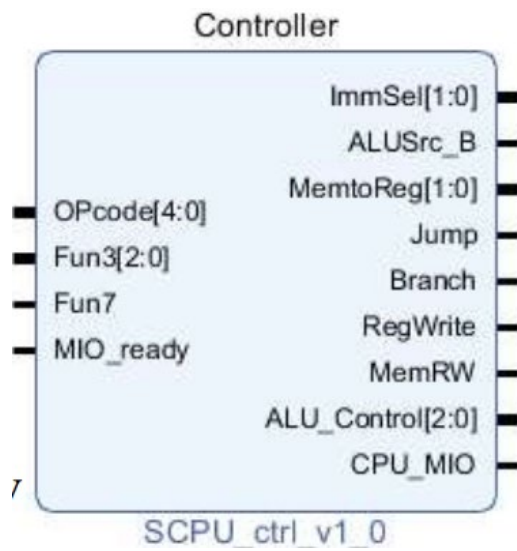
2.1 计算机系统中 CPU 组成



• 数据通路 Data_path 作为 CPU 主要部件之一，具有通用计算功能的算术逻辑部件（主要为 ALU）、具有通用目的寄存器堆、具有通用计数所需的尽可能的路径。



• 控制器 SCPU_ctrl 是寄存器传输控制技术中的运算和通路控制器，它起到指令译码（inst_in 的分块）、产生操作控制信号（ALU 运算控制）、产生指令所需的路径选择的作用。

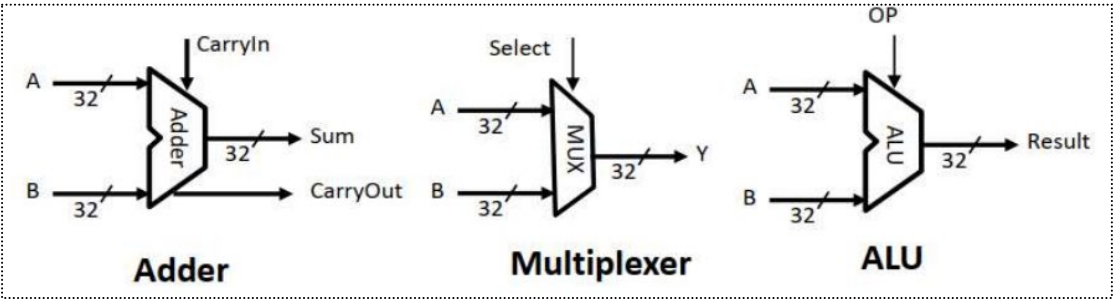


2.2 RISC-V 数据通路 Datapath 原理介绍

2.2.1 数据通路组成部分

包含了完成处理器所要求的的操作所必须的硬件，包括运算单元、寄存器组、状态寄存器等

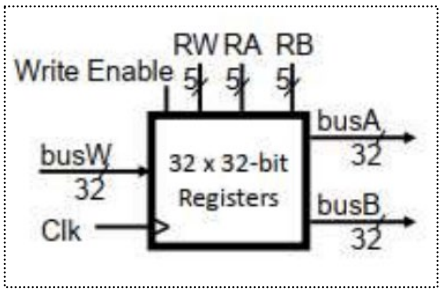
(1) 组合逻辑单元：加法器、多路选择器、ALU 运算单元、立即数生成模块 ImmGen



Instruction type	Instruction opcode[6:0]	Instruction operation	(sign-extend)immediate	Imm Sel
I-type	0000011	Lw;lb;lh;lb;lh	(sign-extend) instr[31:20]	00
	0010011	Addi;slti;sltiu;xori;ori;andi;		
	1100111	jair		
S-type	0100011	Sw;sb;sh	(sign-extend) instr[31:25],[11:7]	01
B-type	1100011	Beq;bne;blt;bge;bltu;bgeu	(sign-extend) instr[31],[7],[30:25],[11:8],1'b0	10
J-type	1101111	jal	(sign-extend) instr[31],[19:12],[20],[30:21],1'b0	11
2024/3/21			Chapter 6	19

ImmGen 利用后续 SCPU_ctrl 给出的 ImmSel 为不同格式的指令生成 32 位立即数（前二十位是符号位拓展，见上图）。

(2) 时序逻辑单元：寄存器堆（Register Files）、存储器（Data Memory）



Register Files 由 32 个 register 组成，包括 1 个 32 位数据输入端口（RegWrite）、2 个 32 位数据输出端口（Rs1_data、Rs2_data）。

(3) 寄存器类型分组：

Register(s)	Alt.	Description
x0	zero	The zero register, always zero
x1	ra	The return address register, stores where functions should return
x2	sp	The stack pointer, where the stack ends
x5-x7, x28-x31	t0-t6	The temporary registers
x8-x9, x18-x27	s0-s11	The saved registers
x10-x17	a0-a7	The argument registers, a0-a1 are also return value

2.2.2 RISC-V 指令

(1) RISC-V 基本指令格式：

寄存器-寄存器操作的 R 型指令；短立即数和访存 load 操作的 I 型指令；访存 store 操作的 S 型指令；条件跳转操作的 B 类型指令；长立即数的 U 型指令和无条件跳转的 J 型指令。

如下图给出了各个指令的编码规则：

31	30	25	24	21	20	19	15	14	12	11	8	7	6	0			
funct7				rs2			rs1		funct3		rd			opcode		R-type	
imm[11:0]						rs1		funct3		rd			opcode		I-type		
imm[11:5]				rs2			rs1		funct3		imm[4:0]			opcode		S-type	
imm[12]		imm[10:5]			rs2			rs1		funct3		imm[4:1]		imm[11]		opcode	B-type
imm[31:12]										rd			opcode		U-type		
imm[20]		imm[10:1]			imm[11]			imm[19:12]			rd			opcode		J-type	

- **opcode**：操作码，用于表示指令格式和指令操作的字段
- **rs1**：只读。第 1 个源操作数寄存器，寄存器地址（编号）是 0x00~0x1F；
- **rs2**：只读。第 2 个源操作数寄存器，寄存器地址（编号）是 0x00~0x1F；
- **rd**：只写。目的操作数寄存器，寄存器地址（编号）是 0x00~0x1F；
- **funct3/7**：功能码，在指令中用来指定指令的功能与操作码配合使用；
- **immediate**：立即数，用作无符号的逻辑操作数、有符号的算术操作数、数据加载（Load）/数据保存（Store）指令的数据地址字节偏移量和分支指令中相对程序计数器（PC）的有符号偏移量；

【对于同一指令格式中的指令来说，数据通路、opcode 基本相同并通用，主要通过 funct3/7 来进行功能决定】

(2) 本 Datapath 实现的指令：

R : add , sub , and , or , xor , srl , slt ;

I : addi , andi , ori , xori , srli , slti , lw ;

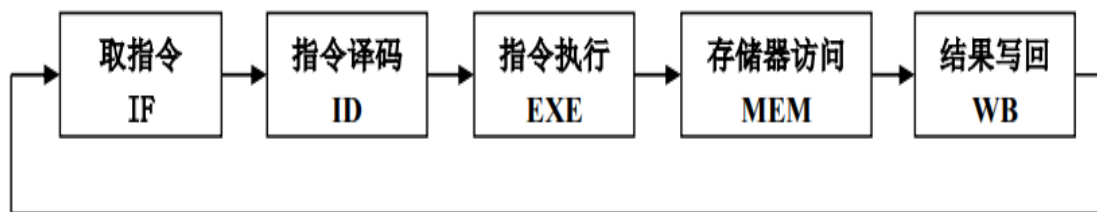
S : sw ; B : beq ; J : jal

Lab4-03 中需要拓展以上 5 类指令，并添加 U 型指令。

2.3 单周期 CPU 简单介绍

单周期 CPU 指的是一条指令的执行在一个时钟周期内完成，然后开始下一条指令的执行，即一条指令用一个时钟周期完成。

我们讨论的单周期主要包括以下五个阶段：



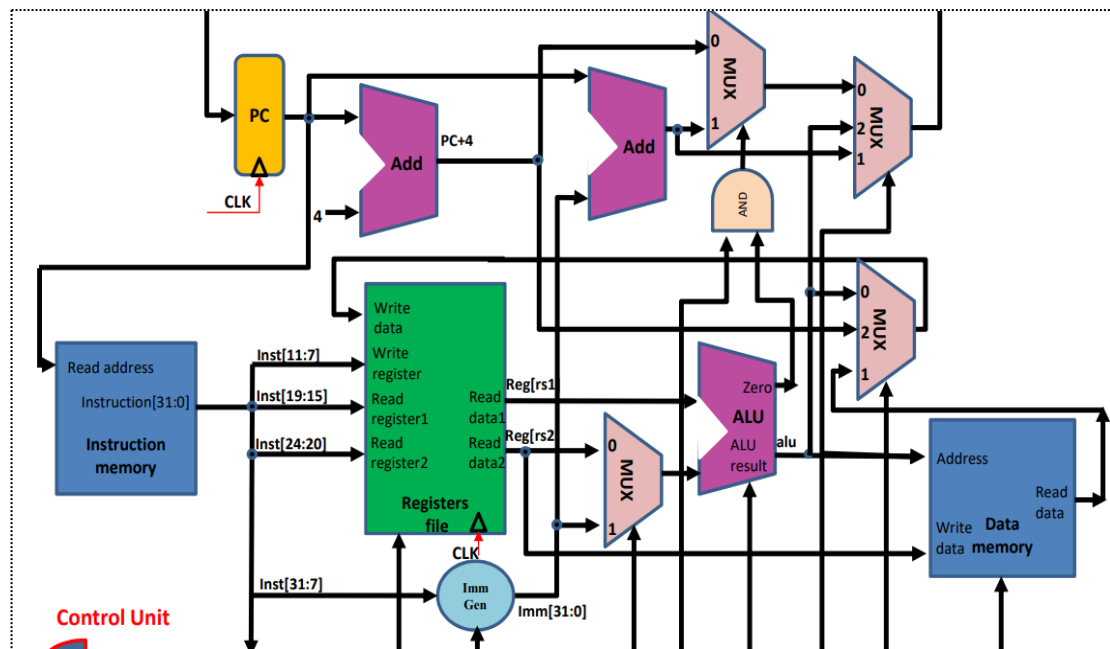
(1) **取指令(IF)**：根据程序计数器 PC 中的指令地址，从存储器中取出一条指令，同时，PC 根据指令字长度自动递增产生下一条指令所需要的指令地址，但遇到“地址转移”指令时，则控制器把“转移地址”送入 PC。

(2) **指令译码(ID)**：对取指令操作中得到的指令进行分析并译码，确定这条指令需要完成的操作，从而产生相应的操作控制信号，用于驱动执行状态中的各种操作。

(3) **指令执行(EXE)**：根据指令译码得到的操作控制信号，具体地执行指令动作，然后转移到结果写回状态。

(4) **存储器访问(MEM)**：所有需要访问存储器的操作都将在这个步骤中执行，该步骤给出存储器的数据地址，把数据写入到存储器中数据地址所指定的存储单元或者从存储器中得到数据地址单元中的数据。

(5) **结果写回(WB)**：指令执行的结果或者访问存储器中得到的数据写回相应的目的寄存器中。



【单周期 CPU 数据通路结构】

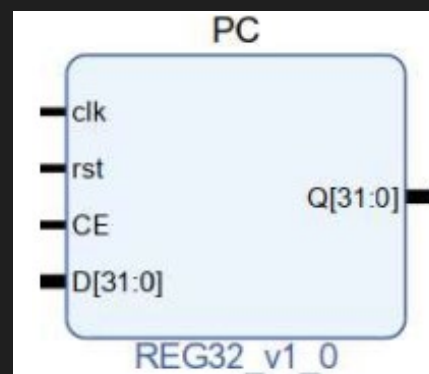
- Instruction Memory: 指令存储器,
- Registers file: 寄存器组
- ALU: 算术逻辑单元 (zero 常用于跳转指令)
- Data Memory: 数据存储器,
- PC: 程序计数器 (存放指令地址)
- ImmGen : 立即数生成单元

2.4 部分新增模块介绍

(1) PC 寄存器 (REG32.v):

用于指令地址的锁存, 采用行为描述设计。

```
module REG32(
    input clk, // 上升沿触发
    input rst,
    input CE,
    input [31:0]D,
    output reg [31:0]Q);
    always @(posedge clk or posedge rst)
        if (rst==1)
            Q <= 32'b0;
        else if (CE == 1)
            Q <= D;
endmodule
```

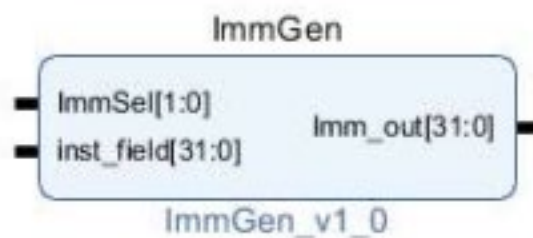


(2) ImmGen 立即数生成模块:

根据输入指令产生立即数的逻辑功能，以及转移指令偏移量左移位的功能。

```
`timescale 1ns / 1ps
module ImmGen(
    input wire [1:0] ImmSel, // 立即数操作控制
    input wire [31:0] inst_field, // Imm 指令数据范围【31: 7】
    output reg [31:0] Imm_out // 立即数输出
);

always@*begin
    case(ImmSel)
2'b00:Imm_out = {{20{inst_field[31]}},inst_field[31:20]}; //addi\lw(I)
2'b01:Imm_out = {{20{inst_field[31]}},inst_field[31:25],inst_field[11:7]}; //sw(s)
2'b10:Imm_out =
{{20{inst_field[31]}},inst_field[7],inst_field[30:25],inst_field[11:8],1'b0};
//beq(b)
2'b11:Imm_out =
{{12{inst_field[31]}},inst_field[19:12],inst_field[20],inst_field[30:21],1'b0};
//jal(j)
    endcase
    end
endmodule
```



三、实验过程和数据记录

3.1 设计实现 Datapath:

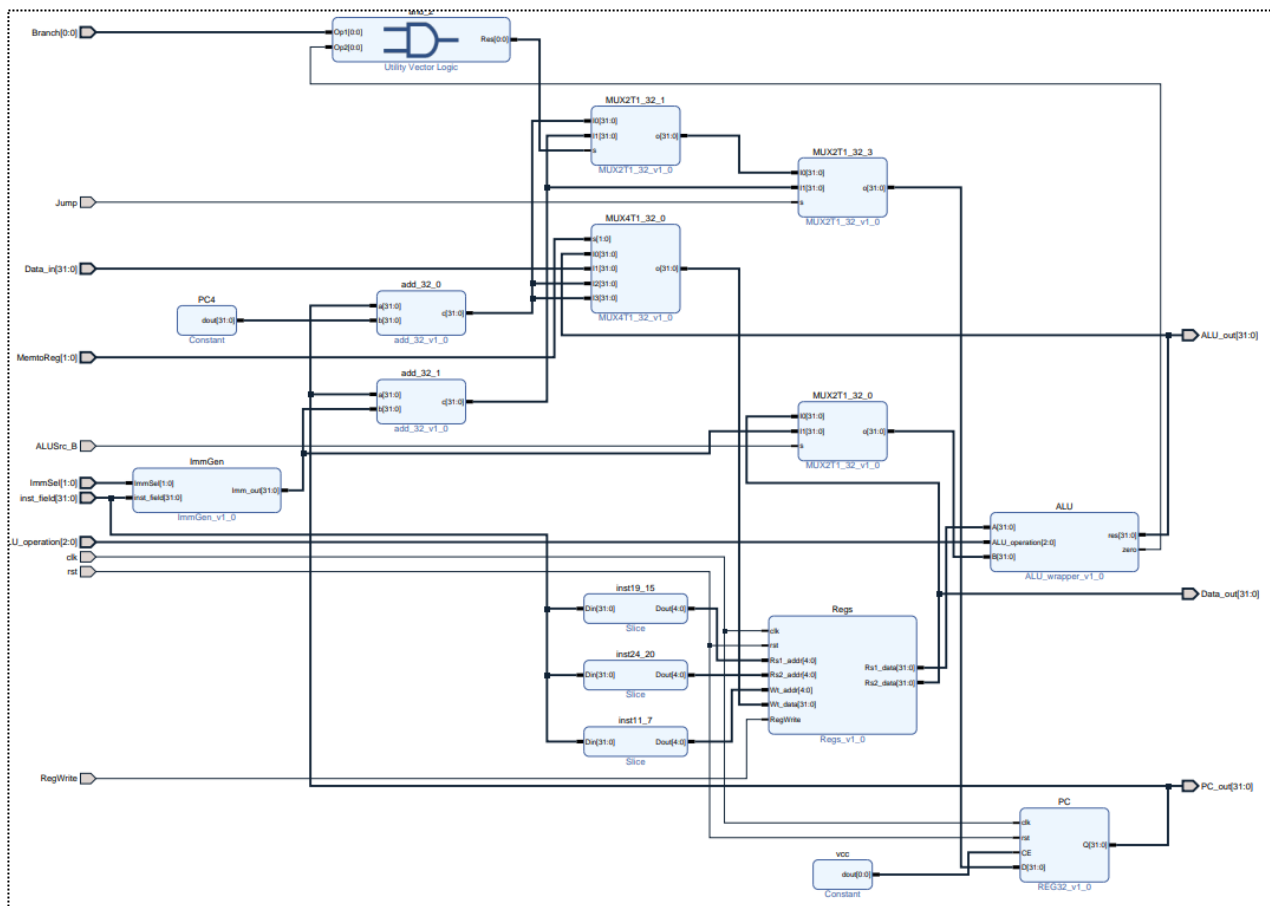
(1) Vivado 新建工程

- 利用 Vivado 新建工程，命名 OExp04-Datapath ；
- 最后正确选择 FPGA 芯片——xc7a100tcsg324-1；

本实验需要用到 Lab0 设计的 MUX 多路选择器、若干基本运算模块(and、or、ADC、xor、nor、srl 等)，以及 Lab1 设计的 ALU、Regs。

同时需要创建 PC、ImmGen 两个模块（具体代码见实验原理）。

(2) 设计 CPU 中的 Datapath:



逻辑原理图如上，代码实现如下：

```
`timescale 1ns / 1ps

module DataPath(
    input wire clk,
    input wire rst,
    input wire[31:0] inst_field,
    input wire[31:0] Data_in,
    input wire[2:0] ALU_operation,
    input wire[1:0] ImmSel,
    input wire[1:0] MemtoReg,
    input wire ALUSrc_B,
    input wire Jump,
    input wire Branch,
    input wire RegWrite,
    output wire[31:0] PC_out,
    output wire[31:0] Data_out,
    output wire[31:0] ALU_out
);

    wire[31:0] Imm_out;
    wire[31:0] PC_inc;
    wire[31:0] PC_imm;
    wire ALU_zero;
    wire[31:0] PC_1;
    wire[31:0] Wt_data;
    wire[31:0] PC_new;
    wire[31:0] Rs1_data;
    wire[31:0] ALU_B;
    ImmGen ImmGen(
        .ImmSel(ImmSel),
        .inst_field(inst_field),
        .Imm_out(Imm_out)
    );
    add32 add_32_0(
        .a(PC_out),
        .b(32'h4),
        .c(PC_inc)
    );
    add32 add_32_1(
        .a(PC_out),
        .b(Imm_out),
        .c(PC_imm)
    );
);
```



```

MUX2T1_32 MUX2T1_32_1(
    .I0(PC_inc),
    .I1(PC_imm),
    .s(Branch & ALU_zero),
    .o(PC_1)
);
MUX4T1_32 MUX4T1_32_0(
    .s(MemtoReg),
    .I0(ALU_out),
    .I1(Data_in),
    .I2(PC_inc),
    .I3(PC_inc),
    .o(Wt_data)
);
MUX2T1_32 MUX2T1_32_3(
    .I0(PC_1),
    .I1(PC_imm),
    .s(Jump),
    .o(PC_new)
);
MUX2T1_32 MUX2T1_32_0(
    .I0(Data_out),
    .I1(Imm_out),
    .s(ALUSrc_B),
    .o(ALU_B)
);
Regs Regs(
    .clk(clk),
    .rst(rst),
    .RegWrite(RegWrite),
    .Rs1_addr(inst_field[19:15]),
    .Rs2_addr(inst_field[24:20]),
    .Wt_addr(inst_field[11:7]),
    .Wt_data(Wt_data),
    .Rs1_data(Rs1_data),
    .Rs2_data(Data_out)
);
ALU ALU(
    .A(Rs1_data),
    .B(ALU_B),
    .ALU_operation(ALU_operation),
    .res(ALU_out),
    .zero(ALU_zero)
);

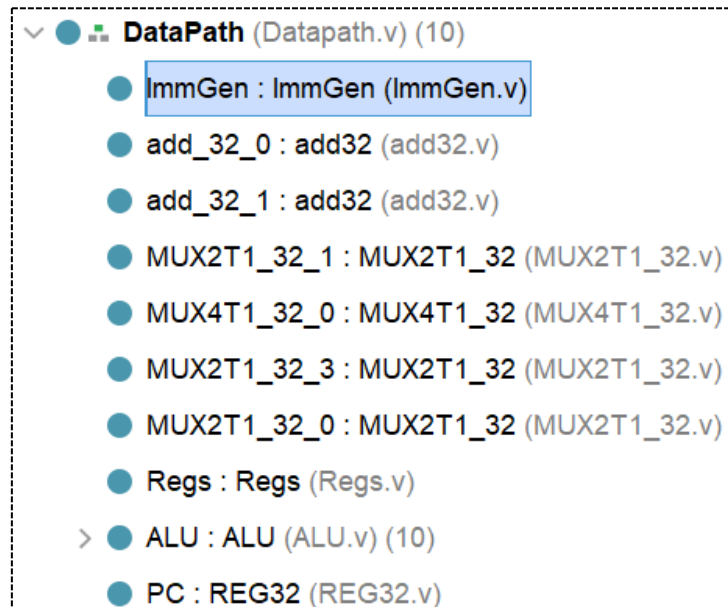
```

```

REG32 PC(
    .clk(clk),
    .rst(rst),
    .CE(1'b1),
    .D(PC_new),
    .Q(PC_out)
);
endmodule

```

(3) Lab04-1Datapath 工程结构:



(4) Datapath 仿真测试:

此处 Datapath 行为仿真，对 I、R、S、B、J 型指令都进行了仿真，同时对 add、sub、xor、beq、jal 等典型指令进行了测试（包含了部分寄存器的遍历）。

仿真代码如下：

```

`timescale 1ns / 1ps
module Datapath_tb();
    reg clk;
    reg rst;
    reg[31:0] inst_field;
    reg[31:0] Data_in;
    reg[2:0] ALU_operation;
    reg[1:0] ImmSel;
    reg[1:0] MemtoReg;
    reg ALUSrc_B;
    reg Jump;
    reg Branch;
    reg RegWrite;

```

```

wire[31:0] PC_out;
wire[31:0] Data_out;
wire[31:0] ALU_out;

DataPath U1(
    .clk(clk),
    .rst(rst),
    .inst_field(inst_field),
    .Data_in(Data_in),
    .ImmSel(ImmSel),
    .MemtoReg(MemtoReg),
    .ALU_operation(ALU_operation),
    .ALUSrc_B(ALUSrc_B),
    .Jump(Jump),
    .Branch(Branch),
    .RegWrite(RegWrite),
    .PC_out(PC_out),
    .Data_out(Data_out),
    .ALU_out(ALU_out)
);

always #5 clk = ~clk;
initial begin
    rst = 1;
    clk = 0;

    Branch = 0;
    Jump = 0;
    ALUSrc_B = 0;
    ALU_operation = 3'b010;
    MemtoReg = 2'b00;
    ImmSel = 2'b00;
    Data_in = 32'b00000000000000000000000000000000;
    RegWrite = 0; #4
    rst = 0;
    //I addi x3,x1,100
    inst_field = 32'b000001100100_00001_000_00011_0010011;
    ALUSrc_B = 1;
    ALU_operation = 3'b010;
    ImmSel = 2'b00;
    RegWrite = 1;
    MemtoReg = 2'b00;
    #10
    //R add x1,x2,x3

```

```

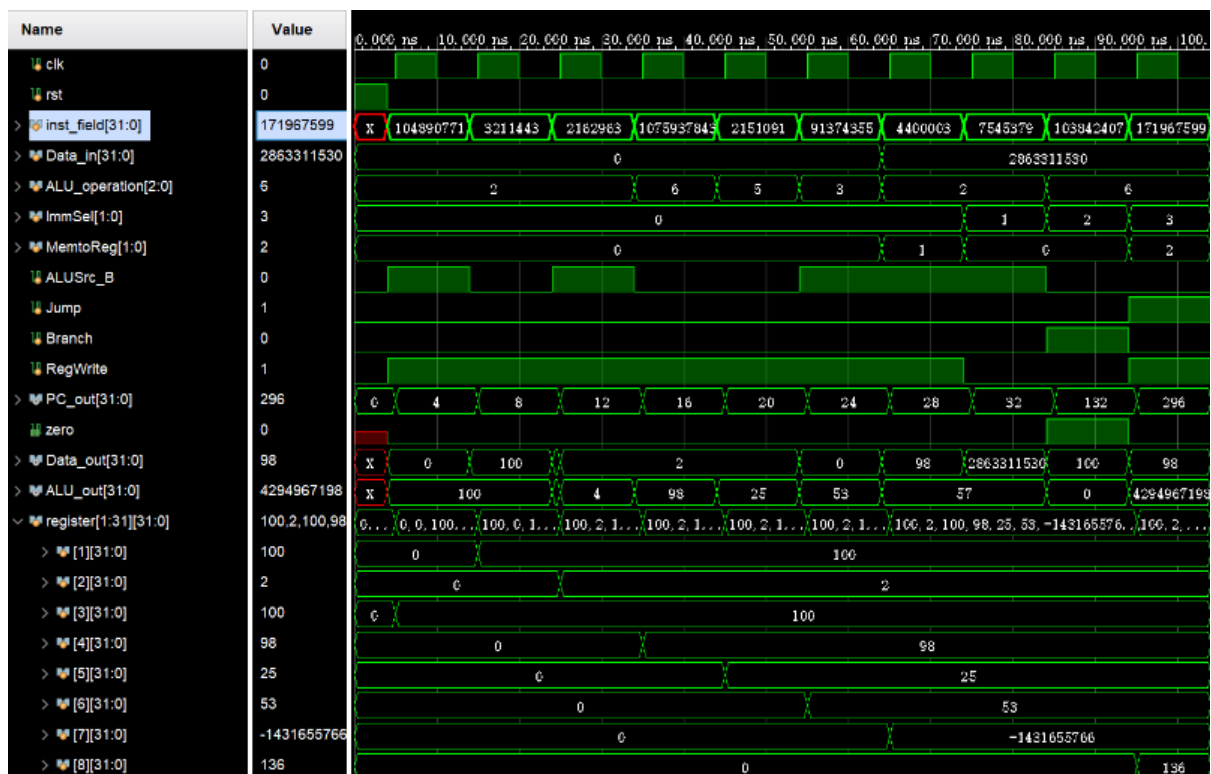
inst_field = 32'b00000000_00011_00010_000_00001_0110011;
ALU_operation = 3'b010;
RegWrite = 1;
ALUSrc_B = 0;
MemtoReg = 2'b00;    #10
//I addi x2,x2,2
inst_field = 32'b0000000000010_00010_000_00010_0010011;
ALUSrc_B = 1;
ALU_operation = 3'b010;
ImmSel = 2'b00;
RegWrite = 1;
MemtoReg = 2'b00;    #10
//R sub x4 x3 x2
inst_field = 32'b0100000_00010_00011_000_00100_0110011;
ALUSrc_B = 0;
ALU_operation = 3'b110;
RegWrite = 1;
MemtoReg = 2'b00;
#10
//R srl x5 x1 x2
inst_field = 32'b00000000_00010_00001_101_00101_0110011;
ALUSrc_B = 0;
ALU_operation = 3'b101;
RegWrite = 1;
MemtoReg = 2'b00;
#10
//I xori x6,x4,87
inst_field = 32'b000001010111_00100_100_00110_0010011;
ALUSrc_B = 1;
ALU_operation = 3'b011;
ImmSel = 2'b00;
RegWrite = 1;
MemtoReg = 2'b00;
#10
//I lw x7,4(x6)
inst_field = 32'b0000000000100_00110_010_00111_0000011;
ALUSrc_B = 1;
ALU_operation = 3'b010;
ImmSel = 2'b00;
RegWrite = 1;
Data_in = 32'b1010_1010_1010_1010_1010_1010_1010_1010;
MemtoReg = 2'b01;
#10
//S sw x7,4(x6)

```

```

inst_field = 32'b0000000_00111_00110_010_00100_0100011;
ALUSrc_B = 1;
ALU_operation = 3'b010;
ImmSel = 2'b01;
RegWrite = 0;
MemtoReg = 2'b00;
#10
//B beq x1,x3,100
inst_field = 32'b0000011_00011_00001_000_00100_1100111;
Branch = 1;
ALUSrc_B = 0;
ALU_operation = 3'b110;
ImmSel = 2'b10;
#10
// J jal x8 164
inst_field = 32'b0000101_00100_00000_000_01000_1101111;
Branch = 0;
Jump = 1;
RegWrite = 1;
MemtoReg = 2'b10;
ImmSel = 2'b11;
#10
$finish;
end
endmodule

```



四、实验结果分析

见上图仿真结果，加以分析（rst 初始化后寄存器值都为 0）：

- (1) `addi x3, x1, 100` (I 型) : $x3 = 100$
- (2) `add x1, x2, x3` (R 型) : $x1 = 0 + 100 = 100$
- (3) `addi x2, x2, 2` (I 型) : $x2 = 2$
- (4) `sub x4, x3, x2` (R 型) : $x4 = 100 - 2 = 98$
- (5) `srl x5, x1, x2` (R 型) : $x5 = 100 \gg 2 = 25$
- (6) `xori x6, x4, 87` (I 型) : $x6 = 1100010 \text{ xor } 1010111 = 0110101 = 53$
- (7) `lw x7, 4(x6)` (I 型) : 仅用来检测数据通路是否正确
- (8) `sw x7, 4(x6)` (S 型) : 仅用来检测数据通路是否正确
- (9) `beq x1, x3, 100` (B 型) : $x1 = x3 = 100$, 因此进行跳转

注意，此时 PC 地址为 32，那么下一次地址 PC_out 为 $32 + 100 = 132$.

Beq 跳转时 `zero = branch = 1`，使下一次 PC 地址选择 PC+100 而非 PC+4.

- (10) `jal x8, 164` (J 型) : $x8 = 132 + 4 = 136$, $PC_out = 132 + 164 = 296$

完成了 `jump` 的指令，并对 x8 进行了赋值。

综上所述，五类指令的正确性基本得到了验证，结果也符合预期，其中对于指令完备性和数据存储器模块（Data Memory）地测试会在 Lab4-03（所有指令均实现）后进行统一验证。

五、讨论与心得

(1) 思考题 1: 扩展下列指令, 数据通路将作如何修改:

R-Type: sra,sll,sltu; **I-Type:** srarai,slli,sltiu ; **B-Type:** bne; **U-Type:** lui;

首先 R 型指令增加且超过了 8 个, 这意味着 3 位的 ALU_Control 不够分配, 需要将 Datapath 中的 ALU_control 修改成 4 位。

增加了 U 型指令 (大立即数), 2 位的 ImmSel 也不足控制, 需要增加到 3 位
此外, 添加 lui 后控制 Write data 的 MUX 需要添加位宽, 使得大立即数能够写回寄存器组。

```
MUX2T1_32 MUX2T1_32_1(  
    .I0(PC_inc),  
    .I1(PC_imm),  
    .s(Branch & ALU_zero),  
    .o(PC_1)  
);
```

同时添加 bne 判定 PC 跳转时, 如上图的 s 控制无法达到要求, 也需要更改。

(2) 思考题 2: 增加 I-Type 算术运算指令是否需要修改本章设计的数据通路:

在 R-Type 增加后的基础上不需要, 但是最终数据通路与本章设计的有所不同。因为 I-Type 指令还是基于 R-Type 的数据通路, 对于 ALU 的依赖, 就不可避免地导致 ALU_Control 需要增加到 4 位。

本次实验旨在用简单的逻辑模块组成 CPU 的重要部件 Datapath, 从底层出发可以发现其中的结构并没有想象中的高端 (也可能我们实现的比较低级)。

亲手搓一个简单的 CPU 还是蛮有趣的。

浙江大学

本科实验报告

课程名称：计算机组成与设计

姓名：张序鸿

学院：竺可桢学院

专业：计算机科学与技术

学号：3230103265

指导教师：杨莹春

2024 年 11 月 15 日

浙江大学实验报告

课程名称：____计算机组成与设计____ 实验类型：____综合____

实验项目名称：____CPU 设计之控制器____

学生姓名：____张序鸿____ 专业：____混合班____ 学号：____3230103265____

指导老师：____杨莹春、潘之杰____ 助教：____张立冬、马致远____

实验地点：____东 4-509____ 实验日期：____2024 年 11 月 20 日____

一、实验目的和要求

1. 复习寄存器传输控制技术
2. 掌握 CPU 的核心：指令执行过程与控制流关系，并设计 Control Unit
3. 设计控制器
4. 学习测试程序的设计

二、实验内容和原理

内容：

任务一：用硬件描述语言设计实现控制器

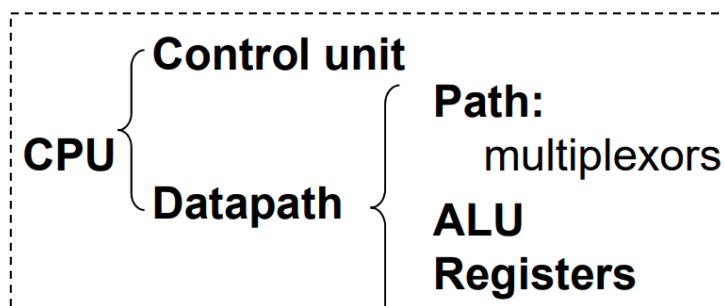
——根据 Exp04-1 数据通路及指令编码完成控制信号真值表

任务二：设计控制器测试方案并完成测试

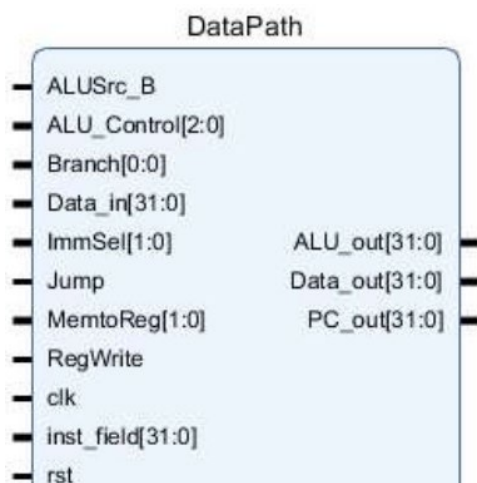
——OP 译码测试：R-格式、访存指令、分支指令，转移指令

原理：

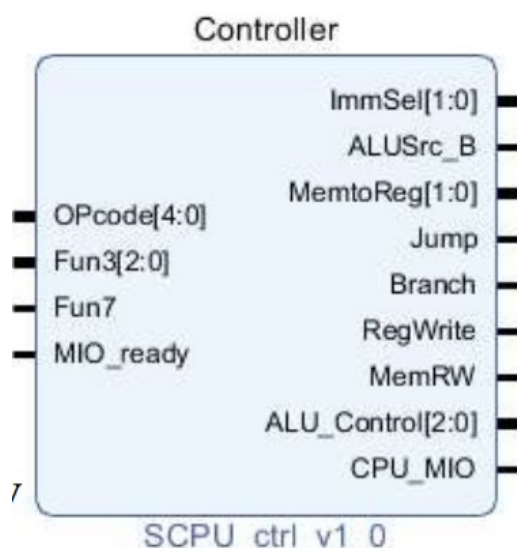
2.1 计算机系统中 CPU 组成



- 数据通路 Data_path 作为 CPU 主要部件之一，具有通用计算功能的算术逻辑部件（主要为 ALU）、具有通用目的寄存器堆、具有通用计数所需的尽可能的路径。



- 控制器 SCPU_ctrl 是寄存器传输控制技术中的运算和通路控制器，它起到指令译码（inst_in 的分块）、产生操作控制信号（ALU 运算控制）、产生指令所需的路径选择的作用。



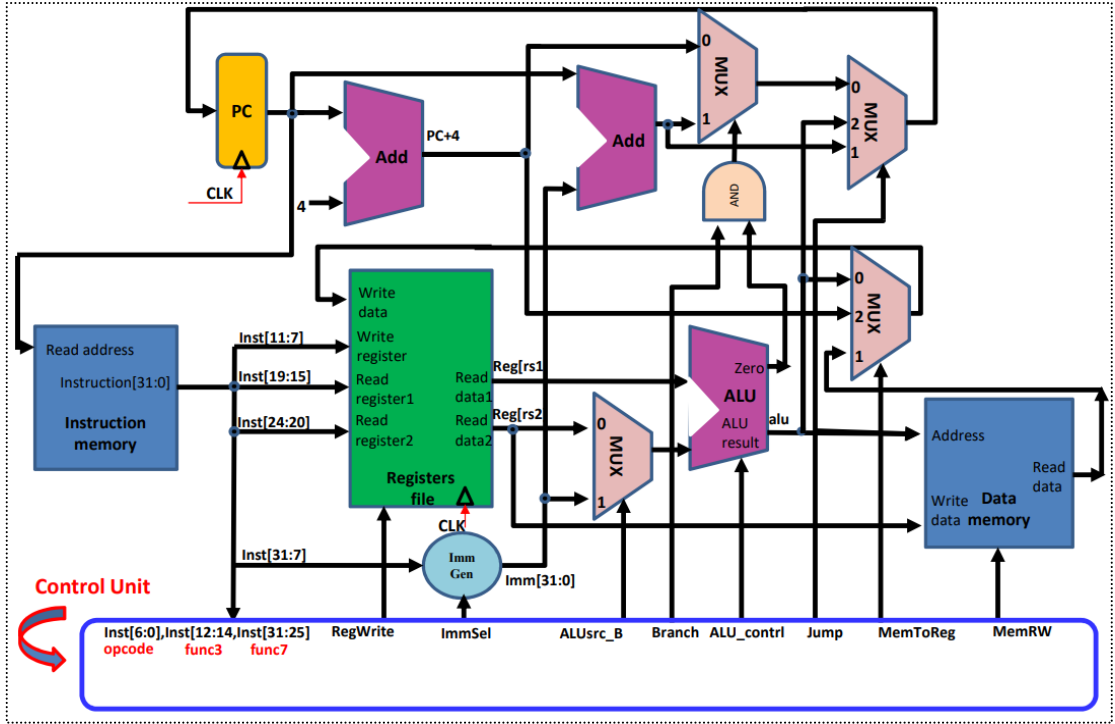
2.2 RISC-V 控制器 SCPU_ctrl 原理介绍

控制单元作为处理器的一部分，用以告诉数据通路需要做什么，包含了取指单元（PC 及地址计算单元）、译码单元、控制单元（时序信号形成及微操作控制信号形成电路）、中断等。

通路和操作控制信号如下：

信号	源数目	功能定义	赋值0时动作	赋值1时动作	赋值2时动作
ALUSrc_B	2	ALU端口B输入选择	选择源操作数寄存器2数据	选择32位立即数（符号扩展后）	-
MemToReg	3	寄存器写入数据选择	选择ALU输出	选择存储器数据	选择PC+4
Branch	2	Beq指令目标地址选择	选择PC+4地址	选择转移目的地址PC+imm（zero=1）	-
Jump	3	J指令目标地址选择	由Branch决定输出	选择跳转目标地址PC+imm（JAL）	-
RegWrite	-	寄存器写控制	禁止寄存器写	使能寄存器写	-
MemRW	-	存储器读写控制	存储器读使能，存储器写禁止	存储器写使能，存储器读禁止	-
ALU_Control	000-111	3位ALU操作控制	参考表ALU_Control		
ImmSel	000-111	3位立即数组合控制	参考表ImmSel		

【具体操作指令的具体信号译码在此不给出，详见后续实验部分】



上图给出了各控制信号在数据通路中的控制线路和作用。

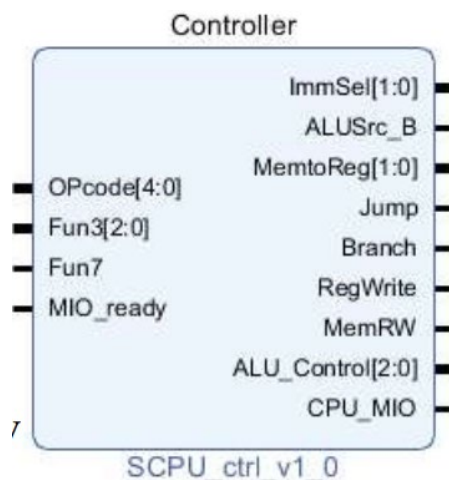
三、实验过程和数据记录

3.1 设计实现 SCPU_Ctrl:

(1) Vivado 新建工程

- 利用 Vivado 新建工程，命名 OExp04-SCPU_ctrl；
- 最后正确选择 FPGA 芯片——xc7a100tcsq324-1；

本实验主要实现两个阶段的译码：根据指令 inst_in 对主要控制信号进行赋值分配；根据 ALU_op、Funct3/7 对 ALU 中的 ALU_Control 进行赋值。



(2) 描述设计 SCPU_ctrl:

- 逻辑接口:

```
module SCPU_ctrl(
    input[4:0]OPcode, //Opcode-----inst[6:2]
    input[2:0]Fun3, //Function-----inst[14:12]
    input Fun7, //Function-----inst[30]
    input MIO_ready, //CPU Wait
    output reg [1:0]ImmSel, //立即数选择控制
    output reg ALUSrc_B, //源操作数 2 选择
    output reg [1:0]MemtoReg, //写回数据选择控制
    output reg Jump, //jal
    output reg Branch, //beq
    output reg RegWrite, //寄存器写使能
    output reg MemRW, //存储器读写使能
    output reg [2:0]ALU_Control, //alu 控制
    output reg CPU_MIO
);
reg [1:0] ALUop;
wire [3:0] Fun;
reg [10:0]CPU_ctrl_signals;
```

- 主控制器设计:

Inst[31:0]	Banch	Jump	ImmSel	ALUSrc_B	ALU_Control	MemRW	RegWrite	MemtoReg
add	0	0	*	Reg	Add	Read	1	ALU
sub	0	0	*	Reg	Sub	Read	1	ALU
(R-R Op)	0	0	*	Reg	(Op)	Read	1	ALU
addi	0	0	I	Imm	Add	Read	1	ALU
lw	0	0	I	Imm	Add	Read	1	Mem
sw	0	0	S	Imm	Add	Write	0	*
beq	0	0	B	Reg	Sub	Read	0	*
beq	1	0	B	Reg	sub	Read	0	*
jal	0	1	J	Imm	*	Read	1	PC+4
lui	0	0	U	Imm	Add	Read	1	ALU

【各类型指令主控制信号真值表】

```

`define CPU_ctrl_signals
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,Jump,ALUop,ImmSel}
    always @(*) begin
        case(OPcode)
            5'b01100: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,Jump,ALUop,ImmSel} =
11'b0_00_1_0_0_0_10_00; end //ALU    - R
            5'b00000: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,Jump,ALUop,ImmSel} =
11'b1_01_1_0_0_0_00_00; end //load    - I
            5'b01000: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,Jump,ALUop,ImmSel} =
11'b1_00_0_1_0_0_00_01; end //store    - S
            5'b11000: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,Jump,ALUop,ImmSel} =
11'b0_00_0_0_1_0_01_10; end //beq     - B
            5'b11011: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,Jump,ALUop,ImmSel} =
11'b0_10_1_0_0_1_00_11; end //jump    - J
            5'b00100: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,Jump,ALUop,ImmSel} =
11'b1_00_1_0_0_0_11_00; end //ALU    - I
            default:
begin {ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,Jump,ALUop,ImmSel} =
11'b000000000000; end
        endcase
    end

```

- ALU 译码控制设计:

Instruction opcode	op	Instruction operation	Funct 3	Funct7	ALUop	Desired ALU action	ALUControl
R-type	0110011	add	000	0000000	10	add	010
		sub	000	0100000		sub	110
		sll	001	0000000		sll	-
		slt	010	0000000		slt	111
		slltu	011	0000000		slltu	-
		xor	100	0000000		xor	011
		srl	101	0000000		srl	101
		sra	101	0100000		sra	-
		or	110	0000000		or	001
		and	111	0000000		and	000

ALU 相关操作对应的控制信号如上图:

```
assign Fun = {Fun3, Fun7};

always @(*) begin
    case(ALUop)
        2'b00: ALU_Control = 3'b010 ; //add
        2'b01: ALU_Control = 4'b110 ; //sub
        2'b10:
            case(Fun)
                4'b0000: ALU_Control = 3'b010 ; //add
                4'b0001: ALU_Control = 3'b110 ; //sub
                4'b1110: ALU_Control = 3'b000 ; //and
                4'b1100: ALU_Control = 3'b001 ; //or
                4'b0100: ALU_Control = 3'b111 ; //slt
                4'b1010: ALU_Control = 3'b101 ; //srl
                4'b1000: ALU_Control = 3'b011 ; //xor
                default: ALU_Control = 3'bx;
            endcase
        2'b11:
            case(Fun3)
                3'b000: ALU_Control = 3'b010 ; //addi
                3'b010: ALU_Control = 3'b111 ; //slti
                3'b100: ALU_Control = 3'b011 ; //xori
                3'b110: ALU_Control = 3'b001 ; //ori
                3'b111: ALU_Control = 3'b000 ; //andi
                3'b101: ALU_Control = 3'b101 ; //srli
                default: ALU_Control = 3'bx ;
            endcase
        endcase
    endcase
end
```

(3) SCPU_ctrl 仿真测试:

SCPU_ctrl 行为仿真, 对 I、R、S、B、J 型指令的控制信号进行了验证, 主要检查对应 Opcode 下各个控制信号是否符合设计要求。

• 仿真代码如下:

```
`timescale 1ns / 1ps

module SCPU_tb();
    reg[4:0] OPcode;
    reg MIO_ready;
    reg [2:0]Fun3;
    reg Fun7;
    reg Fun;
    wire [1:0]ImmSel;
    wire ALUSrc_B;
    wire Jump;
    wire [1:0] MemtoReg;
    wire Branch;
    wire RegWrite;
    wire MemRW;
    wire [2:0]ALU_Control;
    wire CPU_MIO;

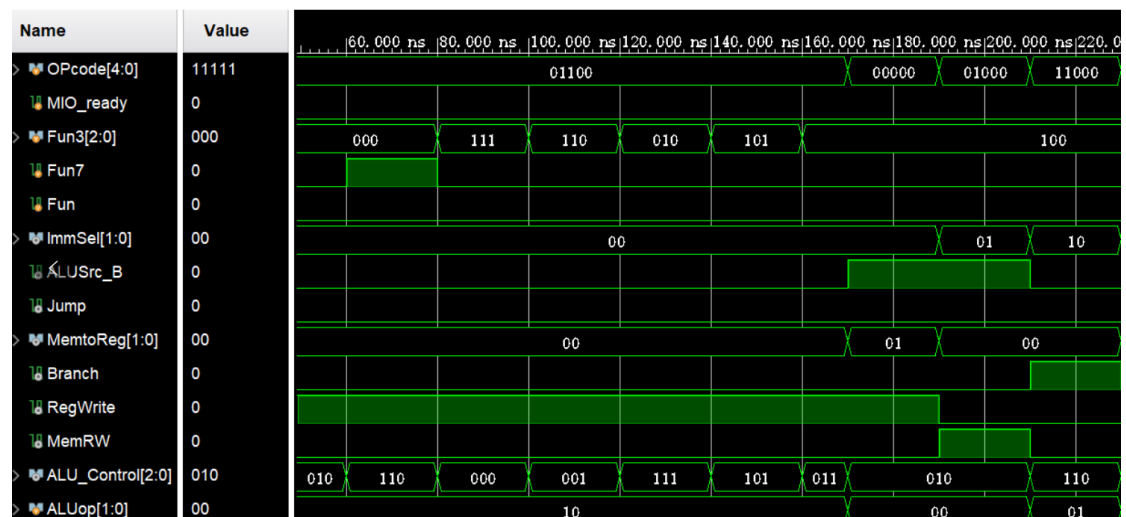
    SCPU_ctrl U1(
        .OPcode(OPcode),
        .Fun3(Fun3),
        .Fun7(Fun7),
        .MIO_ready(MIO_ready),
        .ImmSel(ImmSel),
        .ALUSrc_B(ALUSrc_B),
        .MemtoReg(MemtoReg),
        .Jump(Jump),
        .Branch(Branch),
        .RegWrite(RegWrite),
        .MemRW(MemRW),
        .ALU_Control(ALU_Control),
        .CPU_MIO(CPU_MIO)
    );
endmodule
```

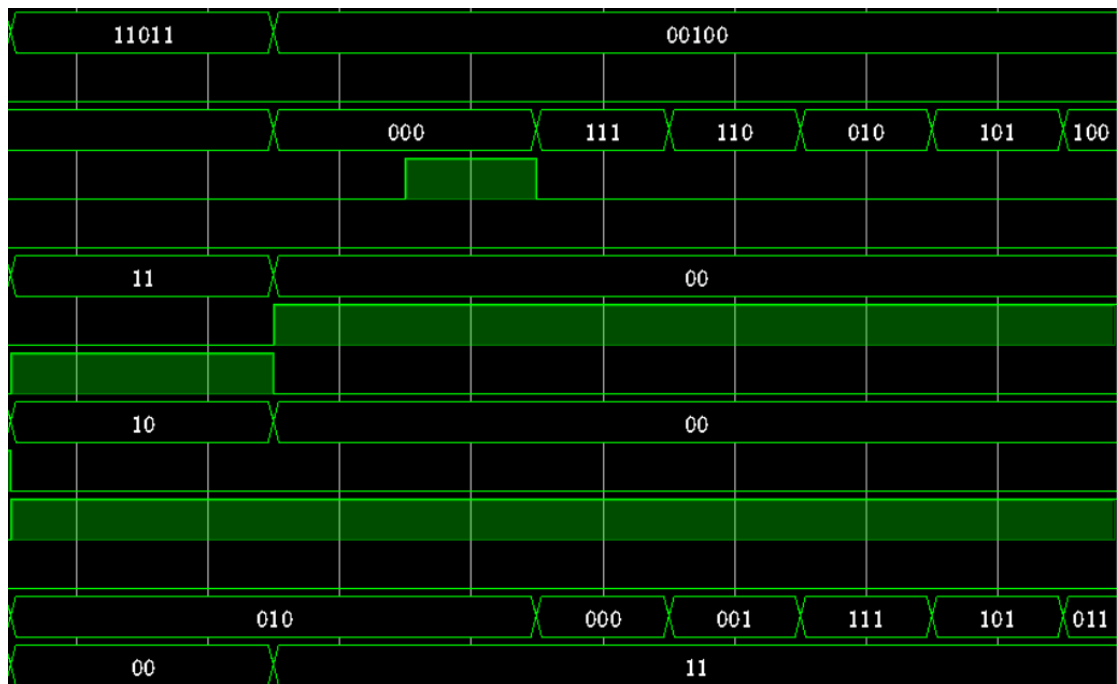
```

initial begin
    OPCODE = 0;
    Fun = 0;
    Fun3 = 0;
    Fun7 = 0;
    MIO_ready = 0; #40; // Wait 40 ns for global reset to finish

    OPCODE = 5'b01100;          //ALU 指令,检查 ALUOp=2'b10; RegWrite=1
    Fun3 = 3'b000; Fun7 = 1'b0;#20; //add,检查 ALU_Control=3'b010
    Fun3 = 3'b000; Fun7 = 1'b1;#20; //sub,检查 ALU_Control=3'b110
    Fun3 = 3'b111; Fun7 = 1'b0;#20; //and,检查 ALU_Control=3'b000
    Fun3 = 3'b110; Fun7 = 1'b0;#20; //or,检查 ALU_Control=3'b001
    Fun3 = 3'b010; Fun7 = 1'b0;#20; //slt,检查 ALU_Control=3'b111
    Fun3 = 3'b101; Fun7 = 1'b0;#20; //sr1,检查 ALU_Control=3'b101
    Fun3 = 3'b100; Fun7 = 1'b0;#10; //xor,检查 ALU_Control=3'b011
    OPCODE = 5'b00000; #20//load 指令,检查 ALUOp=2'b00// ALUSrc_B=1,
MemtoReg=1, RegWrite=1
    OPCODE = 5'b01000; #20; //store 指令,检查 ALUOp=2'b00, MemRW=1,
ALUSrc_B=1
    OPCODE = 5'b11000; #20//beq 指令,检查 ALUOp=2'b01, Branch=1 #20;
    OPCODE = 5'b11011; #40//jump 指令,检查 Jump=1
    OPCODE = 5'b00100; //I 指令,检查 ALUOp=2'b11; RegWrite=1 #20;
    Fun3 = 3'b000; Fun7 = 1'b0;#20; //addi,检查 ALU_Control=3'b010
    Fun3 = 3'b000; Fun7 = 1'b1;#20; //subi,检查 ALU_Control=3'b110
    Fun3 = 3'b111; Fun7 = 1'b0;#20; //andi,检查 ALU_Control=3'b000
    Fun3 = 3'b110; Fun7 = 1'b0;#20; //ori,检查 ALU_Control=3'b001
    Fun3 = 3'b010; Fun7 = 1'b0;#20; //slti,检查 ALU_Control=3'b111
    Fun3 = 3'b101; Fun7 = 1'b0;#20; //srli,检查 ALU_Control=3'b101
    Fun3 = 3'b100; Fun7 = 1'b0;#20; //xori,检查 ALU_Control=3'b011
    OPCODE = 5'h1f; //间隔
    Fun3 = 3'b000; Fun7 = 1'b0; //间隔
end

```

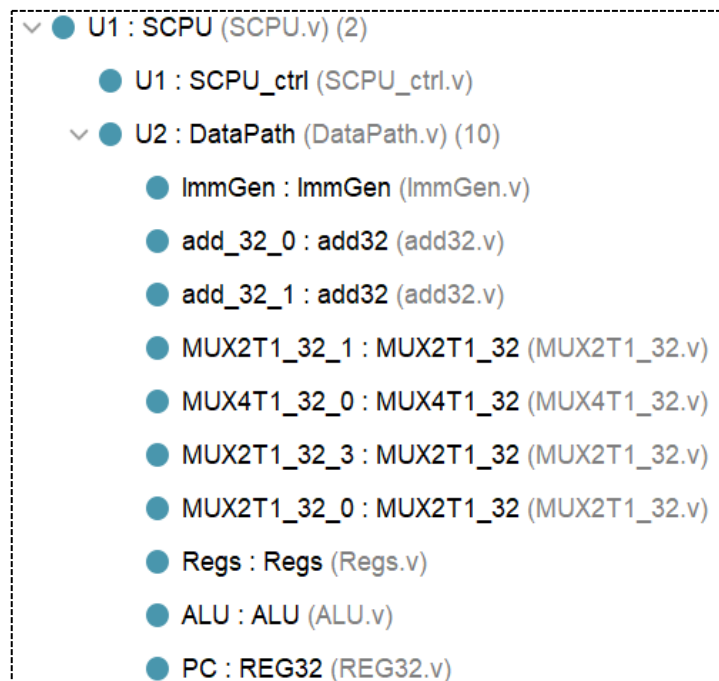




控制信号的具体赋值在仿真代码中注释，可以看到仿真结果与预期结果一致，SCPU_ctrl 的设计正确。

3.2 集成 CPU 并搭建仿真平台

通过 Lab4-01 和 Lab4-02 的实验，我们通过逻辑代码实现了 Datapath 和 SCPU_ctrl 模块，因此我们可以集成得到一个 CPU。



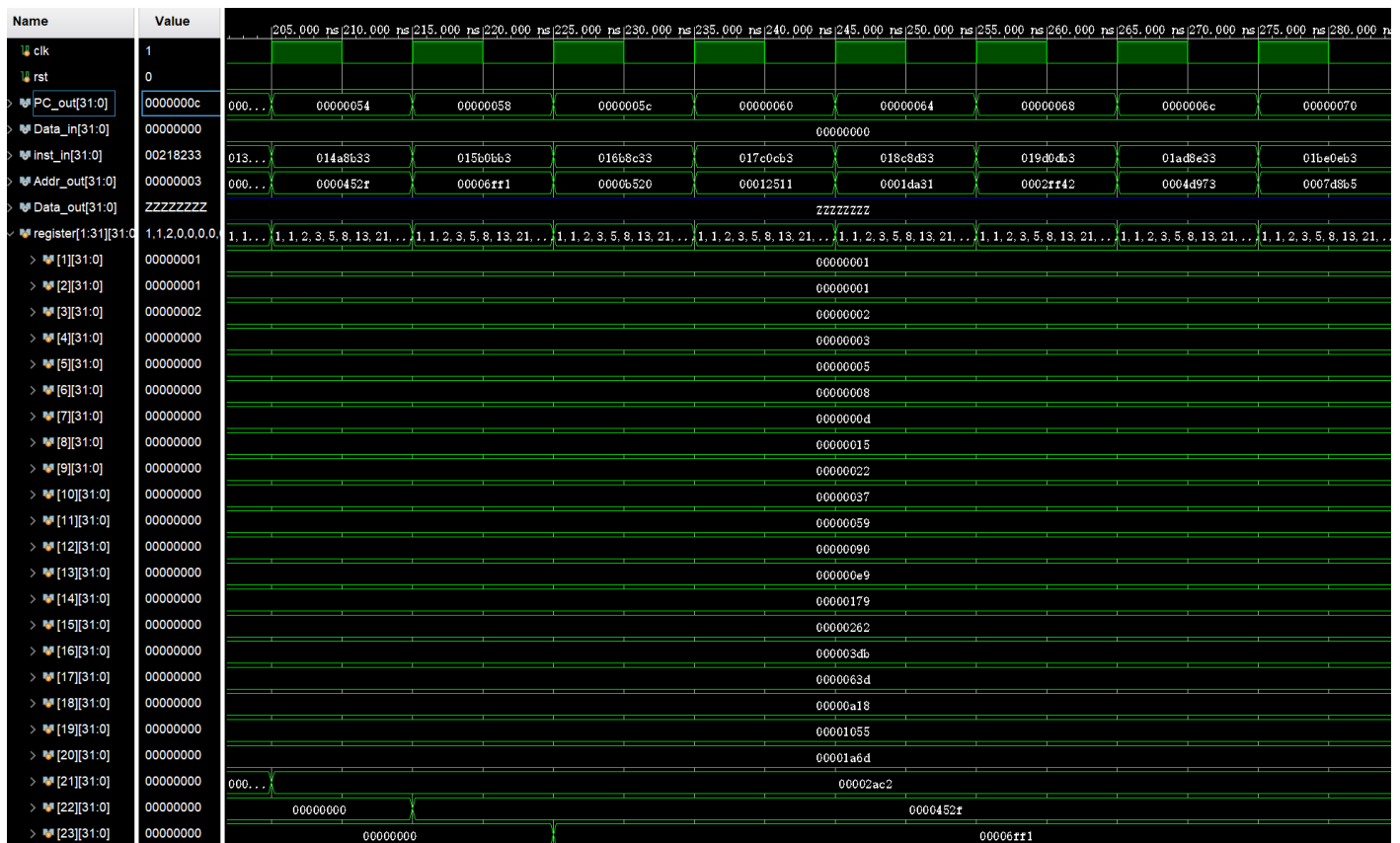
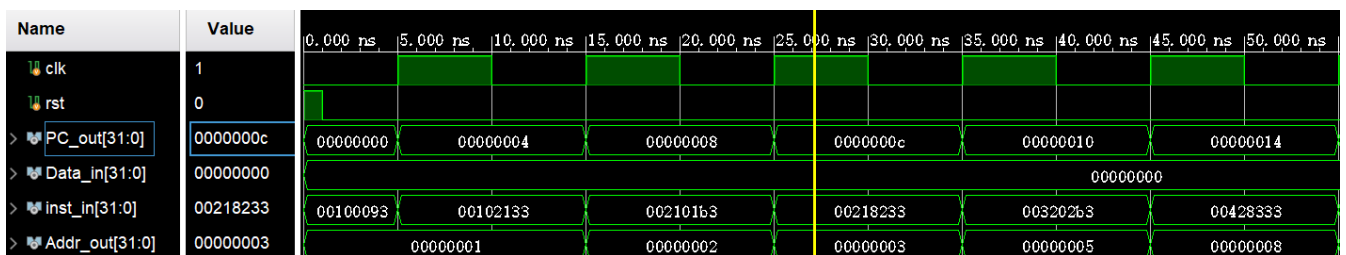
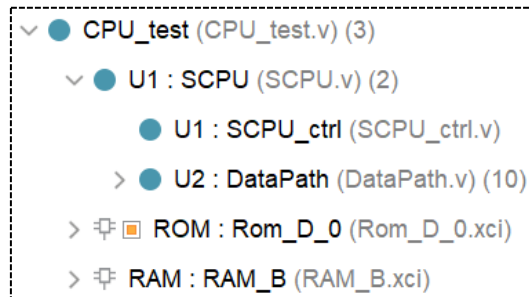
【具体代码如下】

```

module SCPU(
    input clk,
    input rst,
    input [31:0]Data_in,
    input [31:0]inst_in,
    input MIO_ready,
    output MemRW,
    output CPU_MIO,
    output [31:0]Data_out,
    output [31:0]PC_out,
    output [31:0]Addr_out
);
    wire [2:0]ALU_Control;
    wire [1:0]ImmSel , MemtoReg;
    wire Jump, Branch, RegWrite, ALUSrc_B;
    SCPU_ctrl U1(
        .OPcode(inst_in[6:2]),
        .Fun3(inst_in[14:12]),
        .Fun7(inst_in[30]),
        .MIO_ready(MIO_ready),
        .ImmSel(ImmSel),
        .ALUSrc_B(ALUSrc_B),
        .MemtoReg(MemtoReg),
        .Jump(Jump),
        .Branch(Branch),
        .RegWrite(RegWrite),
        .ALU_Control(ALU_Control),
        .CPU_MIO(CPU_MIO) );
    DataPath U2(
        .ALUSrc_B(ALUSrc_B),
        .ALU_operation(ALU_Control),
        .Branch(Branch),
        .Data_in(Data_in),
        .ImmSel(ImmSel),
        .Jump(Jump),
        .MemtoReg(MemtoReg),
        .RegWrite(RegWrite),
        .clk(clk),
        .inst_field(inst_in),
        .rst(rst),
        .ALU_out(Addr_out),
        .Data_out(Data_out),
        .PC_out(PC_out));
endmodule

```

利用 CPU（本实验设计）、RAM、ROM 可以搭建如下图的仿真平台，初始化 RAM 的数据（D_mem.coe），初始化 ROM 的指令（I_mem.coe），可以完成 CPU 各种类型指令的执行，查看 CPU 数据的输入和输出，以及底层的包括寄存器在内的个模块的状态。



四、实验结果分析

CPU 集成时采用的仿真指令仍旧采用了 Lab2 中的 I_mem.coe，主要是为了验证自主集成的 CPU 是否能够进行正常的指令读取与简单操作（指令完备性在 ReadmeLab4.txt 中提到会在 Lab4-03 的 DEMO 中体现，在此不重复验证）。

3.2 中的图一可以看出 PC 可以正常+4 进行下一指令的读取，同时 addr_out = alu_res 可以看出 ALU 的计算结果。

图二则是对寄存器组的观察，可以看到寄存器组能够正常更新。

从仿真结果来看，Lab4-00/01/02 共同实现的简单 CPU 的功能没有问题！

五、讨论与心得

思考题 1：单周期控制器时序体现在哪里？

“always@后面内容是敏感变量，always@()里面的敏感变量为*，意思是说敏感变量由综合器根据 always 里面的输入变量自动添加。”*

```
always @(*) begin
    case(OPcode)
    endcase
always @(*) begin
    case(ALUop)
    endcase
end
```

单周期控制器中采用二级译码，第一级的敏感变量是 Opcode（唯一输入），而且 Opcode 是每时序周期根据指令修改，因此完成一级译码。

二级译码中的 ALUop 则是一级译码的结果，也是在每一周期中进行判断，从而输出 ALU_Control。

思考题 2：设计 bne 指令需要增加控制信号吗？

需要添加一个类似 Branch 的控制信号 BranchN。

Beq 判定跳转的条件是 zero & Branch = 1（利用一个 and），如果不添加控制信号，Branch = 1、zero = 0 无法实现 bne 跳转。

因此添加 BranchN，在判定 PC 跳转的 MUX 处，添加一个 or，使得 BranchN = 1、zero = 0 时也能够利用 BranchN & (! zero) 来判定 bne 的跳转。

思考题 3:

(1) 扩展下列指令, 控制器将作如何修改: **R-Type: sra, sll, sltu; I-Type: srar, sllr, sltur ; B-Type: bne; U-Type: lui; ?**

增加 R-Type、I-Type 时, 由于指令超过 8 个, ALU_Control 需要修改为 4 位, 因此第二级 ALU 译码时需要修改。

增加了 B-Type、U-Type, 因此需要在译码时增加 BranchN 信号、同时修改主控制器 (一级译码), 利用新的 Opcode 来编码 U-Type。

(2) 此时利用二级译码有优势吗?

二级译码让增加新的指令、以及修改译码模式都较轻松。

Lab4 到本实验为止, 完成了简单 CPU 的设计与集成, 设计的时候比较麻烦的是组件模块与线路略微复杂, 导致会出现一些小错误的出现, 需要注意。

但是。。。指令拓展和中断处理还在后头, 需要对 Datapath 和 SCPU_ctrl 的设计过程比较熟悉, 否则在添加新的信号时容易摸不着头脑。

浙江大学

本科实验报告

课程名称：计算机组成与设计

姓名：张序鸿

学院：竺可桢学院

专业：计算机科学与技术

学号：3230103265

指导教师：杨莹春

2024 年 11 月 15 日

浙江大学实验报告

课程名称: 计算机组成与设计 实验类型: 综合

实验项目名称: CPU 设计之指令拓展

学生姓名: 张序鸿 专业: 混合班 学号: 3230103265

指导老师: 杨莹春、潘之杰 助教: 张立冬、马致远

实验地点: 东 4-509 实验日期: 2024 年 11 月 20 日

一、实验目的和要求

1. 复习寄存器传输控制技术
2. 掌握 CPU 的核心: 指令执行过程与控制流关系
3. 拓展设计 Datapath 和 Control Unit
4. 设计测试程序

二、实验内容和原理

内容:

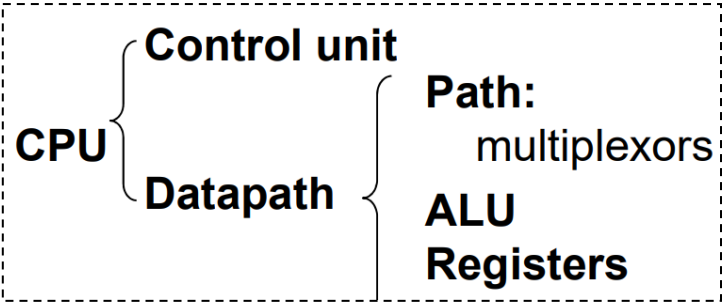
任务一: 重新设计数据通路和控制器

- 兼容 Lab4-1/2 中的数据通路与控制器
- 拓展指令

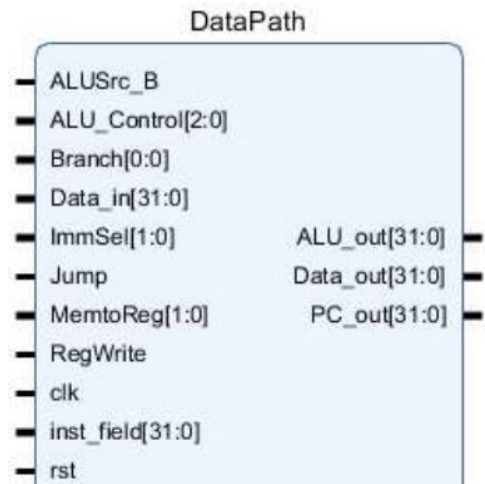
任务二: 设计指令集测试方案并完成测试

原理：

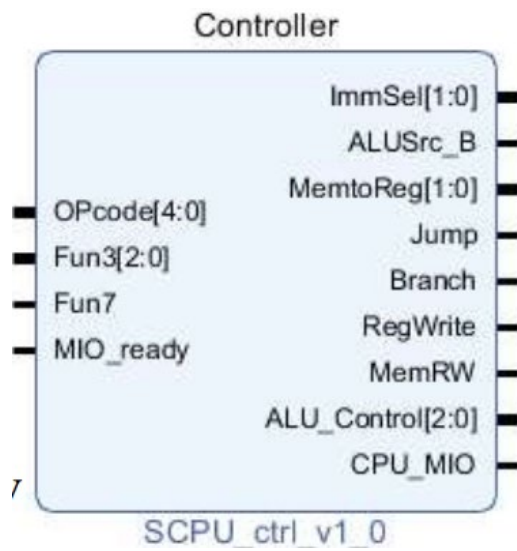
2.1 计算机系统中 CPU 组成



• 数据通路 Data_path 作为 CPU 主要部件之一，具有通用计算功能的算术逻辑部件（主要为 ALU）、具有通用目的寄存器堆、具有通用计数所需的尽可能的路径。



• 控制器 SCPU_ctrl 是寄存器传输控制技术中的运算和通路控制器，它起到指令译码（inst_in 的分块）、产生操作控制信号（ALU 运算控制）、产生指令所需的路径选择的作用。



2.2 RISC-V 指令拓展

前两实验中的 Datapath 和 SCPU_ctrl 实现了 R、I、S、B、J 型指令中的若干指令（黑）：

R : add , sub , and , or , xor , srl , slt ; **sltu , sra , sll**

I : addi , andi , ori , xori , srli , slti , lw ; **sltiu , srai , slli , jalr**

S : sw

B : beq , **bne**

J : jal

Lab4-03 中需要拓展以上 5 类指令，并添加 U-Type 中的 **lui**。

jalr rd, (immediate) rs1 无条件跳转-链接（寄存器地址）

功能: $rd = pc+4$, $pc \leftarrow rs1 + \text{immediate}$

(1) jalr (I-type):

$Rd = PC + 4$ ，写回 Registers file 的数据需添加 PC+4，即为控制 Write data 的 MUX 控制信号 MemtoReg 添加位宽（后面的 lui 也要增加）。

$PC = rs1 + \text{immediate}$ 由 ALU_res 得出，现在下一条 PC 的可能取值有 3 种，因此需要添加 Jump 信号的位宽。

lui rd, immediate

功能: $rd = \text{immediate} \ll 12$

Eg: lui x5,0x12345

$X5 = 0x12345000$ 取左移12位的20位立即数

(2) lui (U-type):

大立即数生成意味着 ImmGen 的控制信号 ImmSel 需要添加新的译码类型。

$Rd = \text{immediate}$ 由 ImmGen 得出，即为控制 Write data 的 MUX 控制信号 MemtoReg 添加位宽。

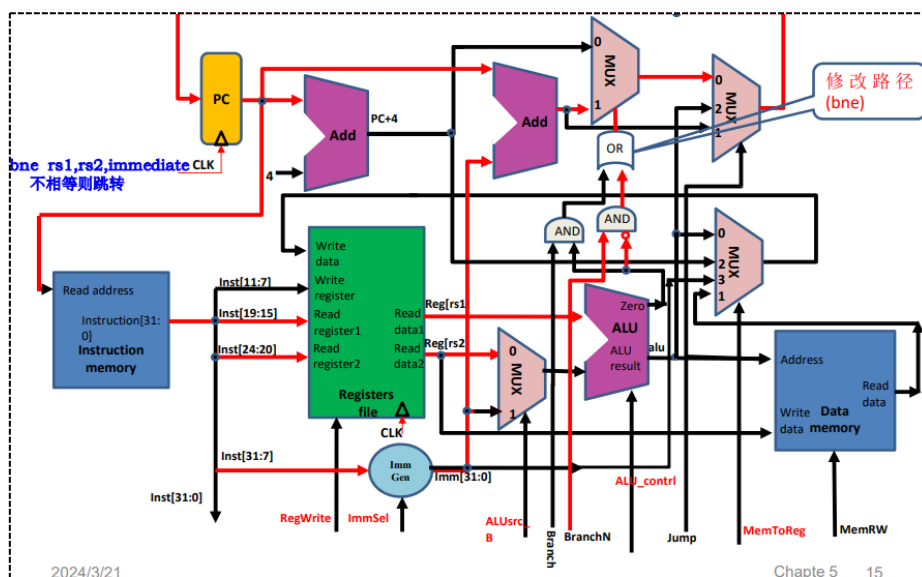
(3) bne (B-Type):

【Bne rs1 , rs2 , immediate （当 rs1 != rs2 时 ， 跳转到 PC + immediate）】

首先需要添加一个类似 Branch 的控制信号 BranchN。

Beq 判定跳转的条件是 zero & Branch = 1 （利用一个 and），如果不添加控制信号，Branch = 1 、 zero = 0 无法实现 bne 跳转。

因此添加 BranchN，在判定 PC 跳转的 MUX 处，添加一个 or，使得 BranchN = 1、zero = 0 时也能够利用 BranchN & (! zero) 来判定 bne 的跳转。



2.3 指令拓展后的数据通路和控制器

2.3.1 New 数据通路:

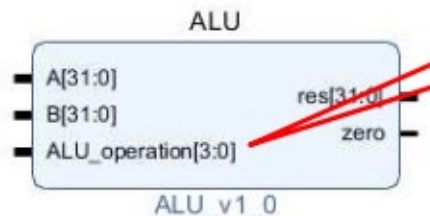
主要的修改端口在 Lab4-01 的思考题与 2.2 中提及。

(1) New 【 ALU 】 :

新添加操作 slt、sra、sll，ALU_operation 位宽变为 4。

```
`timescale 1ns / 1ps
```

```
module ALU(
    input [31:0] A,
    input [3:0] ALU_operation,
    input [31:0] B,
    output reg [31:0] res,
    output zero
);
    parameter one = 32'h00000001, zero_0 = 32'h00000000;
    always @ (*)
```



```

    case (ALU_operation)
    4'b0000: res = A & B; //and
    4'b0001: res = A | B; //or
    4'b0010: res = A + B; //add
    4'b1100: res = A ^ B; //xor
    4'b1101: res = A >> B; //sr1
    4'b0110: res = A - B; //sub
    4'b0111: res = ($signed(A) < $signed(B)) ? one : zero_0; //slt
    4'b1111: res = ($signed(A) >> B); //sra
    4'b1110: res = A << B; //sll
    4'b1001: res = (A < B) ? one : zero_0; //sltu
    default: res = 32'hx;
    endcase
    assign zero = (res==0)? 1: 0;
endmodule

```

(2) New 【 ImmGen 】:

新添加 U-Type,生成一个 20 位立即数,同时 ImmSel 的位宽需要改为 3。

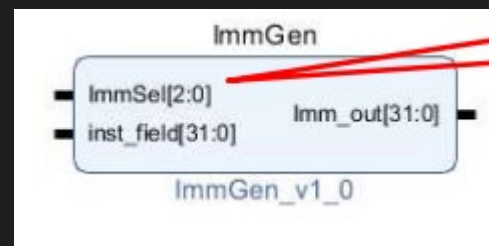
```
`timescale 1ns / 1ps
```

```

module ImmGen_more(
    input wire [2:0] ImmSel,
    input wire [31:0] inst_field,
    output reg [31:0] Imm_out
);

always@*begin
    case(ImmSel)
    3'b000:Imm_out = {inst_field[31:12],12'h000}; // U-type
    3'b001:Imm_out = {{20{inst_field[31]}},inst_field[31:20]}; //addi\lw(I)
    3'b010:Imm_out =
    {{20{inst_field[31]}},inst_field[31:25],inst_field[11:7]}; //sw(s)
    3'b011:Imm_out =
    {{20{inst_field[31]}},inst_field[7],inst_field[30:25],inst_field[11:8],
    1'b0}; //beq(b)
    3'b100:Imm_out =
    {{12{inst_field[31]}},inst_field[19:12],inst_field[20],inst_field[30:21
    ],1'b0}; //jal(j)
    endcase
    end
endmodule

```



2.3.2 New 控制器:

信号	源数目	功能定义	赋值0时动作	赋值1时动作	赋值2时动作	赋值3时动作
ALUSrc_B	2	ALU端口B输入选择	选择源操作数寄存器2数据	选择32位立即数(符号扩展后)	-	-
MemToReg	4	寄存器写入数据选择	选择ALU输出	选择存储器数据	选择PC+4	选择imm
Branch	2	Beq指令目标地址选择	选择PC+4地址	选择转移目的地址PC+imm (zero=1)	-	-
BranchN	2	Bne指令目标地址选择	选择PC+4地址	选择转移目的地址PC+imm (zero=0)	-	-
Jump	3	J指令目标地址选择	由Branch决定输出	选择跳转目标地址PC+imm (JAL)	选择跳转目标地址ALU输出(JALR:rs1+imm)	-
RegWrite	-	寄存器写控制	禁止寄存器写	使能寄存器写	-	-
MemRW	-	存储器读写控制	存储器读使能, 存储器写禁止	存储器写使能, 存储器读禁止	-	-
ALU_Control	0000 - 1111	4位ALU操作控制	参考表ALU_Control			
ImmSel	000-111	3位立即数组合控制	参考表ImmSel			

【控制信号定义】

Inst[31:0]	Banch	Branch N	Jump	ImmSel	ALUSrc_B	ALU_Control	MemRW	RegWrite	MemtoReg
add	0	0	0	*	Reg (0)	Add	Read	1	ALU(0)
sub	0	0	0	*	Reg (0)	Sub	Read	1	ALU(0)
(R-R Op)	0	0	0	*	Reg (0)	(Op)	Read	1	ALU(0)
addi	0	0	0	I	Imm (1)	Add	Read	1	ALU(0)
lw	0	0	0	I	Imm (1)	Add	Read	1	Mem(1)
sw	0	0	0	S	Imm (1)	Add	Write	0	*
beq	0	0	0	B	Reg (0)	sub	*	0	*
beq	1	*	0	B	Reg (0)	sub	*	0	*
bne	0	0	0	B	Reg (0)	sub	*	0	*
bne	*	1	0	B	Reg (0)	sub	*	0	*
jalr	*	*	2	I	Imm (1)	Add	*	1	PC+4(2)
jal	*	*	1	J	Imm (1)	*	*	1	PC+4(2)
lui	0	0	0	U	*	*	*	1	Imm(3)

【控制信号真值表】

Instruction type	Instruction opcode[6:0]	Instruction operation	(sign-extend)immediate	Imm Sel
I-type	0000011	Lw;bu;lh; lb;lhu	(sign-extend) instr[31:20]	001
	0010011	Addi;slli;slti u;xori;ori;a ndi;slli;srai		
	1100111	jalr		
S-type	0100011	Sw;sb;sh	(sign-extend) instr[31:25],[11:7]	010
B-type	1100011	Beq;bne;blt bge;bltu;b geu	(sign-extend) instr[31],[7],[30:25],[11:8], 1'b0	011
J-type	1101111	jal	(sign-extend) instr[31],[19:12],[20],[30:21],1 'b0	100
U-type	0010111	auipc	instr[31:12],12'h000	000
	0110111	lui		

【ImmSel 控制信号】

三、实验过程和数据记录

3.1 设计实现 Datapath:

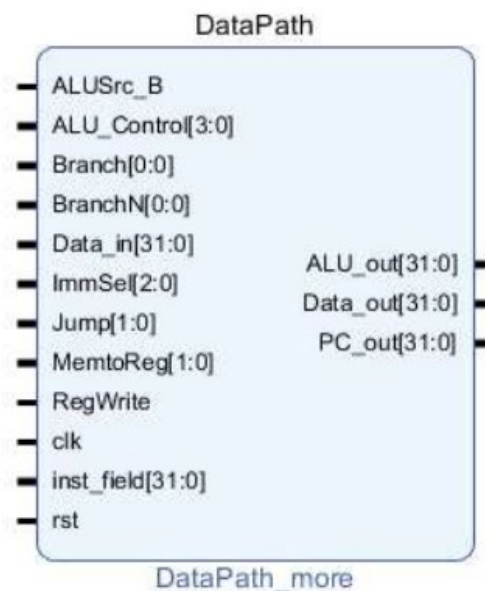
(1) Vivado 新建文件

- 利用 Vivado 新建文件，命名 OExp04-Datapath_more;

本实验仍然需要用到 Lab0 设计的 MUX 多路选择器、若干基本运算模块 (and、or、xor、nor、srl 等)，以及 Lab1 设计的 Regs。

同时需要修改 ALU、ImmGen_more 两个模块 (具体代码见实验原理)。

(2) 设计 CPU 中的 Datapath:



代码实现如下:

```
`timescale 1ns / 1ps
module Datapath_more(
    input wire clk,
    input wire rst,
    input wire[31:0] inst_field,
    input wire[31:0] Data_in,
    input wire[3:0] ALU_operation,
    input wire[2:0] ImmSel,
    input wire[1:0] MemtoReg,
    input wire ALUSrc_B,
    input wire [1:0] Jump,
    input wire Branch, // beq
    input wire BranchN, // bne
    input wire RegWrite,
```

```

output wire[31:0] PC_out,
output wire[31:0] Data_out,
output wire[31:0] ALU_out,
output wire[1023:0] all_regs_data
);

wire[31:0] Imm_out;
wire[31:0] PC_inc;
wire[31:0] PC_imm;
wire ALU_zero;
wire[31:0] PC_1;
wire[31:0] Wt_data;
wire[31:0] PC_new;
wire[31:0] Rs1_data;
wire[31:0] ALU_B;

ImmGen_more ImmGen(
    .ImmSel(ImmSel),
    .inst_field(inst_field),
    .Imm_out(Imm_out)
);

add32 add_32_0(
    .a(32'h4),
    .b(PC_out),
    .c(PC_inc)
);

add32 add_32_1(
    .a(PC_out),
    .b(Imm_out),
    .c(PC_imm)
);

MUX2T1_32 MUX2T1_32_1(
    .I0(PC_inc),
    .I1(PC_imm),
    .s((Branch & ALU_zero) | (BranchN & ~ALU_zero)),
    .o(PC_1)
);

MUX4T1_32 MUX4T1_32_0(
    .s(MemtoReg),
    .I0(ALU_out),
    .I1(Data_in),
    .I2(PC_inc),
    .I3(Imm_out),
    .o(Wt_data)
);

```

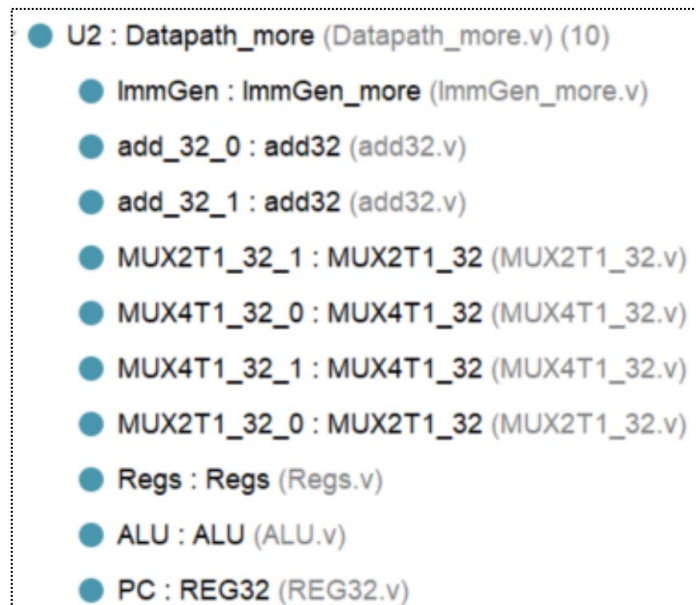
```

);
MUX4T1_32 MUX4T1_32_1(
    .I0(PC_1),
    .I1(PC_imm),
    .I2(ALU_out),
    .I3(PC_1),
    .s(Jump),
    .o(PC_new)
);
MUX2T1_32 MUX2T1_32_0(
    .I0(Data_out),
    .I1(Imm_out),
    .s(ALUSrc_B),
    .o(ALU_B)
);
Regs Regs(
    .clk(clk),
    .rst(rst),
    .RegWrite(RegWrite),
    .Rs1_addr(inst_field[19:15]),
    .Rs2_addr(inst_field[24:20]),
    .Wt_addr(inst_field[11:7]),
    .Wt_data(Wt_data),
    .Rs1_data(Rs1_data),
    .Rs2_data(Data_out),
    .all_regs_data(all_regs_data)
);
ALU ALU(
    .A(Rs1_data),
    .B(ALU_B),
    .ALU_operation(ALU_operation),
    .res(ALU_out),
    .zero(ALU_zero)
);
REG32 PC(
    .clk(clk),
    .rst(rst),
    .CE(1'b1),
    .D(PC_new),
    .Q(PC_out)
);

```

```
endmodule
```

(3) Lab04-3Datapath_more 工程结构:

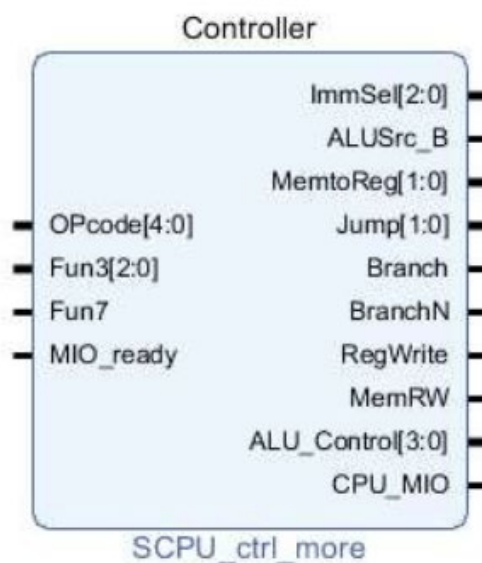


3.2 设计实现 SCPU_Ctrl:

(1) Vivado 新建文件

- 利用 Vivado 新建文件，命名 OExp04-SCPU_ctrl_more;

本实验主要实现两个阶段的译码：根据指令 inst_in 对主要控制信号进行赋值分配；根据 ALU_op、Funct3/7 对 ALU 中的 ALU_Control 进行赋值。



(2) 描述设计 SCPU_ctrl_more:

- 逻辑接口:

```
module SCPU_ctrl_more(  
    input[4:0]OPcode, //Opcode  
    input[2:0]Fun3, //Function  
    input Fun7, //Function  
    input MIO_ready, //CPU Wait  
    output reg [2:0]ImmSel, //立即数选择控制  
    output reg ALUSrc_B, //源操作数 2 选择  
    output reg [1:0]MemtoReg, //写回数据选择控制  
    output reg [1:0]Jump, //jal && jalr  
    output reg Branch, //beq  
    output reg BranchN, // bne  
    output reg RegWrite, //寄存器写使能  
    output reg MemRW, //存储器读写使能  
    output reg [3:0]ALU_Control, //alu 控制  
    output reg CPU_MIO  
);  
reg [1:0] ALUop;  
wire [3:0] Fun;  
assign Fun = {Fun3, Fun7};
```

- 主控制器设计:

```
always @(*) begin  
    case(OPcode)  
5'b01100: begin  
    {ALUSrc_B, MemtoReg, RegWrite, MemRW, Branch, BranchN, Jump, ALUop, ImmSel} =  
    14'b0_00_1_0_0_0_00_10_000; end //ALU  
5'b00000: begin  
    {ALUSrc_B, MemtoReg, RegWrite, MemRW, Branch, BranchN, Jump, ALUop, ImmSel} =  
    14'b1_01_1_0_0_0_00_00_001; end //load  
5'b01000: begin  
    {ALUSrc_B, MemtoReg, RegWrite, MemRW, Branch, BranchN, Jump, ALUop, ImmSel} =  
    14'b1_00_0_1_0_0_00_00_010; end //store  
5'b11000: begin  
    case(Fun3)  
        3'b000: begin  
    {ALUSrc_B, MemtoReg, RegWrite, MemRW, Branch, BranchN, Jump, ALUop, ImmSel} =  
    14'b0_00_0_0_1_0_00_01_011; end // beq  
        3'b001: begin  
    {ALUSrc_B, MemtoReg, RegWrite, MemRW, Branch, BranchN, Jump, ALUop, ImmSel} =  
    14'b0_00_0_0_0_1_00_01_011; end // bne  
    endcase  
    end
```

```

        end
5'b11011: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,BranchN,Jump,ALUop,ImmSel} =
14'b1_10_1_0_0_0_01_00_100; end //jump
5'b00100: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,BranchN,Jump,ALUop,ImmSel} =
14'b1_00_1_0_0_0_00_11_001; end //ALU(addi)
5'b11001: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,BranchN,Jump,ALUop,ImmSel} =
14'b1_10_1_0_0_0_10_00_001; end //jarl
5'b01101: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,BranchN,Jump,ALUop,ImmSel} =
14'b0_11_1_0_0_0_00_00_000; end //lui
default:
begin {ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,BranchN,Jump,ALUop,ImmSel} = 14'h0; end
        endcase
    end
end

```

- ALU 译码控制设计:

```

always @(*) begin
    case(ALUop)
        2'b00: ALU_Control = 4'b0010 ;
        2'b01: ALU_Control = 4'b0110 ;
        2'b10:
            case(Fun)
                4'b0000: ALU_Control = 4'b0010 ; //add
                4'b0001: ALU_Control = 4'b0110 ; //sub
                4'b1110: ALU_Control = 4'b0000 ; //and
                4'b1100: ALU_Control = 4'b0001 ; //or
                4'b0100: ALU_Control = 4'b0111 ; //slt
                4'b1010: ALU_Control = 4'b1101 ; //srl
                4'b1000: ALU_Control = 4'b1100 ; //xor
                4'b0110: ALU_Control = 4'b1001 ; //sltu
                4'b0010: ALU_Control = 4'b1110 ; //sll
                4'b1011: ALU_Control = 4'b1111 ; //sra
                default: ALU_Control = 4'bx;
            endcase
        2'b11:
    end
end

```

```

    case(Fun3)
      3'b000: ALU_Control = 4'b0010 ; //addi
      3'b010: ALU_Control = 4'b0111 ; //slti
      3'b100: ALU_Control = 4'b1100 ; //xori
      3'b110: ALU_Control = 4'b0001 ; //ori
      3'b111: ALU_Control = 4'b0000 ; //andi
      3'b101:
        if(Fun7)
          ALU_Control = 4'b1111 ; //srai
        else
          ALU_Control = 4'b1101 ; //srli
      3'b011: ALU_Control = 4'b1001 ; //sltiu
      3'b001: ALU_Control = 4'b1110 ; //slli
      default: ALU_Control = 4'bx ;
    endcase
  endcase
end

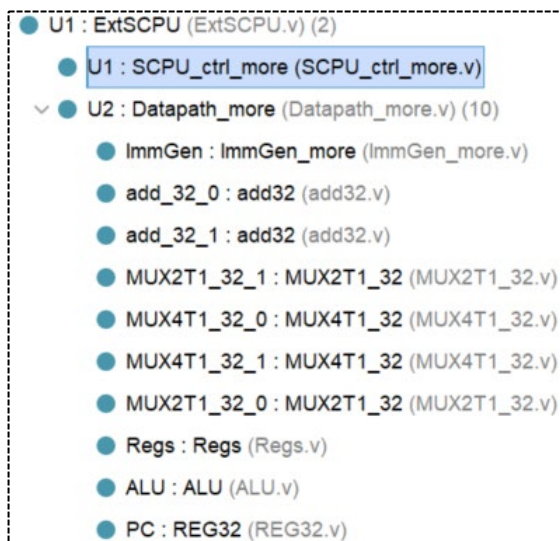
```

3.3 集成 CPU

3.3.1 搭建仿真平台

通过 3.1、3.2，我们通过逻辑代码实现了 Datapath_more 和 SCPU_ctrl_more 模块，对 CPU 进行了指令拓展，我们参考 Lab4-02 的操作也能建立一个 CPU 仿真测试平台，对相关指令进行检测。

首先对两个模块进行集成组合成 CPU：



【具体代码如下】

```
`timescale 1ns / 1ps

module ExtSCPU(
    input clk,
    input rst,
    input [31:0]Data_in,
    input [31:0]inst_in,
    input MIO_ready,

    output MemRW,
    output CPU_MIO,
    output [31:0]Data_out,
    output [31:0]PC_out,
    output [31:0]Addr_out,
    output [1023:0] all_regs_data
);

    wire [3:0]ALU_Control;
    wire [2:0]ImmSel;
    wire [1:0]MementoReg,Jump;
    wire Branch, BranchN, RegWrite, ALUSrc_B;

    SCPU_ctrl_more U1(
        .OPcode(inst_in[6:2]),
        .Fun3(inst_in[14:12]),
        .Fun7(inst_in[30]),
        .MIO_ready(MIO_ready),
        .ImmSel(ImmSel),
        .ALUSrc_B(ALUSrc_B),
        .MementoReg(MementoReg),
        .Jump(Jump),
        .Branch(Branch),
        .BranchN(BranchN),
        .RegWrite(RegWrite),
        .MemRW(MemRW),
        .ALU_Control(ALU_Control),
        .CPU_MIO(CPU_MIO)
    );

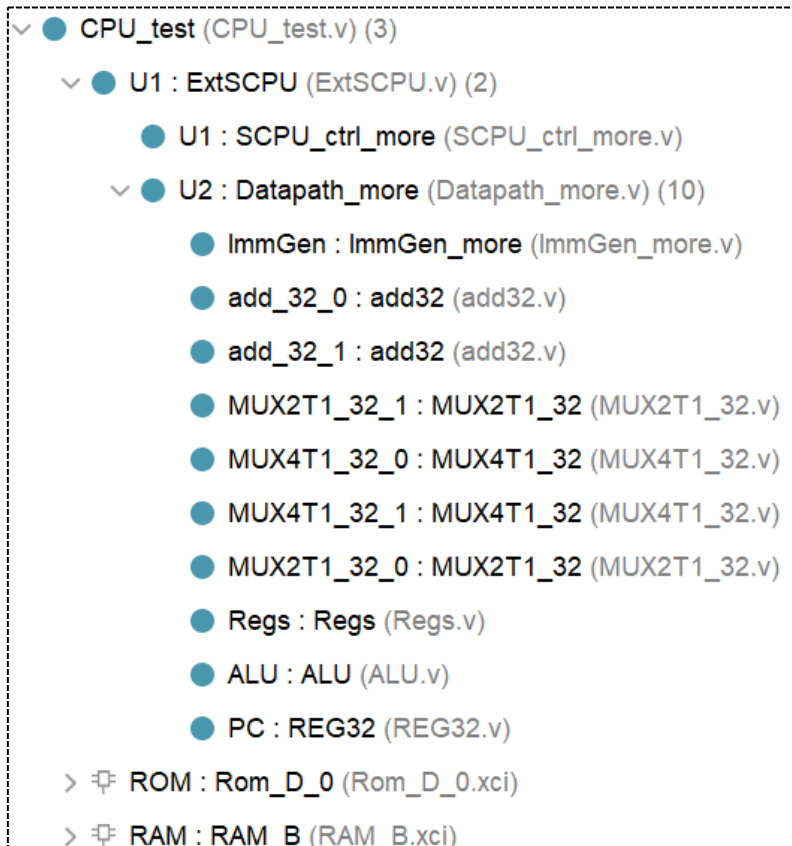
    Datapath_more U2(
        .ALUSrc_B(ALUSrc_B),
        .ALU_operation(ALU_Control),
        .Branch(Branch),
        .BranchN(BranchN),
```

```

        .Data_in(Data_in),
        .ImmSel(ImmSel),
        .Jump(Jump),
        .MemtoReg(MemtoReg),
        .RegWrite(RegWrite),
        .clk(clk),
        .inst_field(inst_in),
        .rst(rst),
        .ALU_out(Addr_out),
        .Data_out(Data_out),
        .PC_out(PC_out),
        .all_regs_data(all_regs_data)
    );
endmodule

```

同样的，利用 CPU（本实验设计）、RAM、ROM 可以搭建如下图的仿真平台，初始化 RAM 的数据(D_mem.coe)，初始化 ROM 的指令(I_mem_more.coe)，查看 CPU 数据的输入和输出，以及底层的包括寄存器在内的各模块的状态。





【此处给出部分仿真结果，对于整套指令的分析见后一下板验证部分】

3.3.2 DEMO 物理测试

继承替换 CPU 及子模块后，下载.bit 文件，使用 DEMO 程序进行物理验证。

本实验由于指令种类多样并且指令较为复杂，通过记录 VGA 中的数据变化，设计了测试记录表格。

- ROM 中填充指令（I_mem_more.coe）如下：

```
memory_initialization_radix=16;
memory_initialization_vector=
00007293, 00007313, 88888137, 00832183, 0032A223, 00402083, 01C02383, 00338863,
555550B7, 0070A0B3, FE0098E3, 007282B3, 00230333, 00531463, 40000033, 40530433,
405304B3, 0080006F, 00007033, 0072F533, 00157593, 00B51463, 00006033, 00A5E5B3,
0015E513, 00558463, 00004033, 00A5C633, 00164613, 00B61463, 00000013, 0012D293,
00060463, 40000033, 00129293, 00B28463, 00000013, 001026B3, 00503733, F65FF06F
```

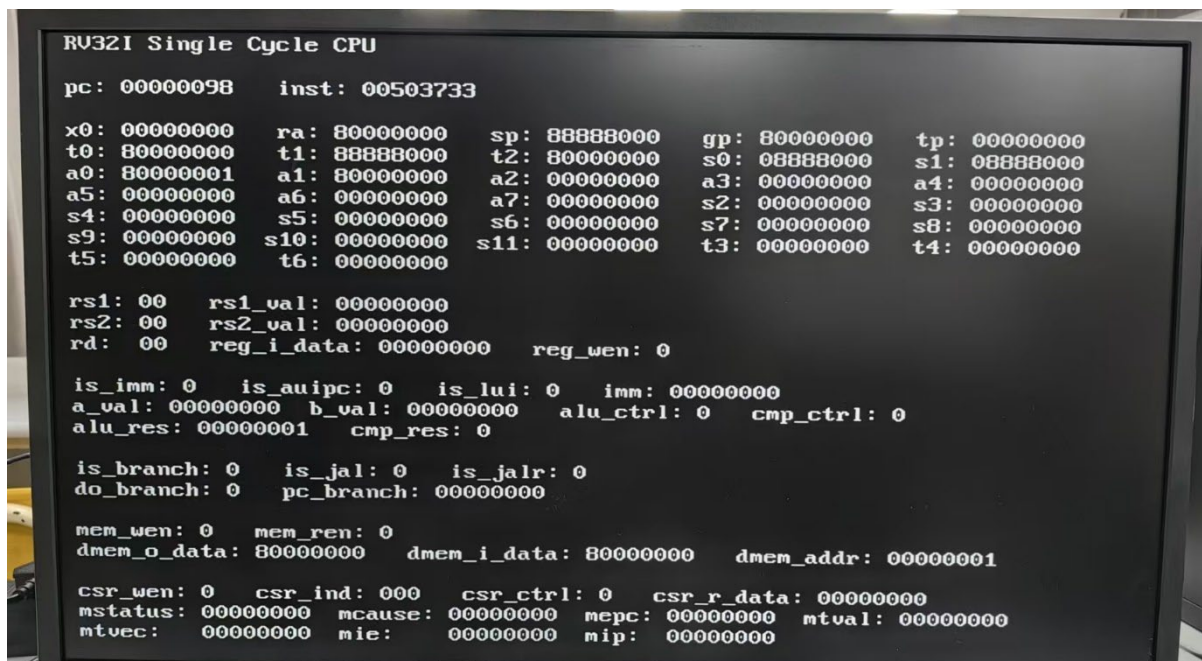
- RAM 中储存的数据（D_mem.coe）如下：

```
memory_initialization_radix=16;
memory_initialization_vector=
F0000000, 000002AB, 80000000, 0000003F, 00000001, FFF70000,
0000FFFF, 80000000, 00000000, 11111111, 22222222, 33333333,
44444444, 55555555, 66666666, 77777777, 88888888, 99999999,
aaaaaaaa, bbbbbbbb, cccccccc, dddddddd, eeeeeeee, FFFFFFFF,
557EF7E0, D7BDFBD9, D7DBFDB9, DFCFFCFB, DFCFBFFF, F7F3DFFF,
FFFFDF3D, FFFF9DB9, FFFBFCFB, DFCFFCFB, DFCFBFFF, D7DB9FFF,
D7DBFDB9, D7BDFBD9, FFFF07E0, 007E0FFF, 03bdf020, 03def820,
08002300
```

部分测试 VGA 显示图片见实验结果分析。

四、实验结果分析

由于指令集较为冗长，此处放了最后 3 个指令的 VGA 显示图片，便于观察之前所有指令所修改的寄存器值是否正确。



【PC = 0x98 inst = 0x00503733 将要执行 sltu x14 , x0 , x5】


```
RV32I Single Cycle CPU
pc: 0000009c    inst: f65ff06f

x0: 00000000    ra: 80000000    sp: 88888000    gp: 80000000    tp: 00000000
t0: 80000000    t1: 88888000    t2: 80000000    s0: 08888000    s1: 08888000
a0: 80000001    a1: 80000000    a2: 00000000    a3: 00000000    a4: 00000001
a5: 00000000    a6: 00000000    a7: 00000000    s2: 00000000    s3: 00000000
s4: 00000000    s5: 00000000    s6: 00000000    s7: 00000000    s8: 00000000
s9: 00000000    s10: 00000000    s11: 00000000    t3: 00000000    t4: 00000000
t5: 00000000    t6: 00000000

rs1: 00    rs1_val: 00000000
rs2: 00    rs2_val: 00000000
rd: 00    reg_i_data: 00000000    reg_wen: 0

is_imm: 0    is_auiopc: 0    is_lui: 0    imm: 00000000
a_val: 00000000    b_val: 00000000    alu_ctrl: 0    cmp_ctrl: 0
alu_res: ffffff64    cmp_res: 0

is_branch: 0    is_jal: 0    is_jalr: 0
do_branch: 0    pc_branch: 00000000

mem_wen: 0    mem_ren: 0
dmem_o_data: 80000000    dmem_i_data: 00000000    dmem_addr: fffffff64

csr_wen: 0    csr_ind: 000    csr_ctrl: 0    csr_r_data: 00000000
mstatus: 00000000    mcause: 00000000    mepc: 00000000    mtval: 00000000
mtvec: 00000000    mie: 00000000    mip: 00000000
```

- (1) 执行完 PC = 0x98 后, x14 = a4 = 0x00000001,
- (2) PC = 0x9c, inst = 0XF65FF06F, 将要执行 jal x0 -156

```
RV32I Single Cycle CPU
pc: 00000000    inst: 00007293

x0: 00000000    ra: 80000000    sp: 88888000    gp: 80000000    tp: 00000000
t0: 80000000    t1: 88888000    t2: 80000000    s0: 08888000    s1: 08888000
a0: 80000001    a1: 80000000    a2: 00000000    a3: 00000000    a4: 00000001
a5: 00000000    a6: 00000000    a7: 00000000    s2: 00000000    s3: 00000000
s4: 00000000    s5: 00000000    s6: 00000000    s7: 00000000    s8: 00000000
s9: 00000000    s10: 00000000    s11: 00000000    t3: 00000000    t4: 00000000
t5: 00000000    t6: 00000000

rs1: 00    rs1_val: 00000000
rs2: 00    rs2_val: 00000000
rd: 00    reg_i_data: 00000000    reg_wen: 0

is_imm: 0    is_auiopc: 0    is_lui: 0    imm: 00000000
a_val: 00000000    b_val: 00000000    alu_ctrl: 0    cmp_ctrl: 0
alu_res: 00000000    cmp_res: 0

is_branch: 0    is_jal: 0    is_jalr: 0
do_branch: 0    pc_branch: 00000000

mem_wen: 0    mem_ren: 0
dmem_o_data: 80000000    dmem_i_data: 00000000    dmem_addr: 00000000

csr_wen: 0    csr_ind: 000    csr_ctrl: 0    csr_r_data: 00000000
mstatus: 00000000    mcause: 00000000    mepc: 00000000    mtval: 00000000
mtvec: 00000000    mie: 00000000    mip: 00000000
```

执行完整套指令, 跳转到 PC = 0 处, 同时寄存器组保存了所有运算值。

PC	Machine	Code Basic	Status	Result
0x0	0x00007293	andi x5 x0 0		x5 = 0
0x4	0x00007313	andi x6 x0 0		x6 = 0
0x8	0x88888137	lui x2 559240		x2 = 0x88888000
0xc	0x00832183	lw x3 8(x6)		x3 = 0x80000000
0x10	0x0032A223	sw x3 4(x5)		Mem[1] = 0x80000000
0x14	0x00402083	lw x1 4(x0)		x1 = 0x80000000
0x18	0x01C02383	lw x7 28(x0)		x7 = 0x80000000
0x1c	0x00338863	beq x7 x3 16		go to 2c
0x20	0x555550B7	lui x1 349525	Jump	
0x24	0x0070A0B3	slt x1 x1 x7	Jump	
0x28	0xFE0098E3	bne x1 x0 -16	Jump	
0x2c	0x007282B3	add x5 x5 x7		x5 = 0x80000000
0x30	0x00230333	add x6 x6 x2		x6 = 0x88888000
0x34	0x00531463	bne x6 x5 8		go to 3c
0x38	0x40000033	sub x0 x0 x0	Jump	
0x3c	0x40530433	sub x8 x6 x5		x8 = 0x08888000
0x40	0x405304B3	sub x9 x6 x5		x9 = 0x08888000
0x44	0x0080006F	jal x0 8		x0 no change
0x48	0x00007033	and x0 x0 x0	Jump	
0x4c	0x0072F533	and x10 x5 x7		x10 = 0x80000000
0x50	0x00157593	andi x11 x10 1		x11 = 0x0
0x54	0x00B51463	bne x10 x11 8		go to 5c
0x58	0x00006033	or x0 x0 x0	Jump	
0x5c	0x00A5E5B3	or x11 x11 x10		x11 = 0x80000000
0x60	0x0015E513	ori x10 x11 1		x10 = 0x80000001
0x64	0x00558463	beq x11 x5 8		go to 6c
0x68	0x00004033	xor x0 x0 x0	Jump	
0x6c	0x00A5C633	xor x12 x11 x10		x12 = 0x00000001
0x70	0x00164613	xori x12 x12 1		x12 = 0x0
0x74	0x00B61463	bne x12 x11 8		go to 7c
0x78	0x00000013	addi x0 x0 0	Jump	
0x7c	0x0012D293	srli x5 x5 1		x5 = 0x40000000
0x80	0x00060463	beq x12 x0 8		go to 88
0x84	0x40000033	sub x0 x0 x0	Jump	
0x88	0x00129293	slli x5 x5 1		x5 = 0x80000000
0x8c	0x00B28463	beq x5 x11 8		go to 94
0x90	0x00000013	addi x0 x0 0	Jump	
0x94	0x001026B3	slt x13 x0 x1		x13 = 0x0 (signed x1 is minus)
0x98	0x00503733	sltu x14 x0 x5		x14 = 0x00000001
0x9c	0xF65FF06F	jal x0 -156		go to 0

上表记录了整套指令的具体操作，相关结果可以对照 Result 和 VGA 显示中寄存器组的相关结果。

该实验成功运行了基础指令和拓展的指令（lui、bne、sltu、srli 等），DEMO 测试程序能够顺利执行，实验结果符合预期。

五、讨论与心得

思考题 1：指令扩展时控制器用二级译码设计存在什么问题？

随着指令拓展，二级译码的中间量的位宽需要不断增加，同时如果要拓展同一指令类型中的指令时，可能存在译码级数增加的可能，如 `srai` 和 `srli` 需要经过 `ALUop---Fun3---Fun7` 的多层连续译码。

```
2'b11:
    case(Fun3)
        3'b000: ALU_Control = 4'b0010 ; //addi
        3'b010: ALU_Control = 4'b0111 ; //slti
        3'b100: ALU_Control = 4'b1100 ; //xori
        3'b110: ALU_Control = 4'b0001 ; //ori
        3'b111: ALU_Control = 4'b0000 ; //andi
        3'b101:
            if(Fun7)
                ALU_Control = 4'b1111 ; //srai
            else
                ALU_Control = 4'b1101 ; //srli
```

思考题 2：设计 `bne` 指令需要增加控制信号吗？

需要添加一个类似 Branch 的控制信号 `BranchN`。

`Beq` 判定跳转的条件是 `zero & Branch = 1`（利用一个 `and`），如果不添加控制信号，`Branch = 1`、`zero = 0` 无法实现 `bne` 跳转。

因此添加 `BranchN`，在判定 PC 跳转的 MUX 处，添加一个 `or`，使得 `BranchN = 1`、`zero = 0` 时也能够利用 `BranchN & (! zero)` 来判定 `bne` 的跳转。

思考题 2：设计 `srai` 时需要增加新的数据通道吗？

`Srai` 作为 I-Type 中的一项指令，需要 `ImmGen` 来生成立即数，并递交给 ALU 进行右移操作，因此不需要添加新的数据通路。

指令拓展实验看似不难，只需要修改数据通路，并更新控制器中的控制信号。但是战线比较长，导致会有小错误出现难以纠正，因此仿真还是比较重要的。

最终下板能看到正确结果（一样的代码两次跑 bit 下板结果不一样？），还是比较惊喜的，革命尚未成功，同志还需努力！

浙江大学

本科实验报告

课程名称：计算机组成与设计

姓 名：张序鸿

学 院：竺可桢学院

专 业：计算机科学与技术

学 号：3230103265

指导教师：杨莹春

2024 年 11 月 15 日

浙江大学实验报告

课程名称: 计算机组成与设计 实验类型: 综合

实验项目名称: CPU 设计之中断

学生姓名: 张序鸿 专业: 混合班 学号: 3230103265

指导老师: 杨莹春、潘之杰 助教: 张立冬、马致远

实验地点: 东 4-509 实验日期: 2024 年 11 月 27 日

一、实验目的和要求

1. 深入理解 CPU 结构
3. 学习如何提高 CPU 使用效率
3. 学习 CPU 中断工作原理
4. 设计中断测试程序
5. 熟悉 RISC-V 中断的原理, 了解引起 CPU 中断产生的原因及其处理方法, 扩展包含中断的 CPU

二、实验内容和原理

内容:

任务一: 扩展实验 CPU 中断功能

- 修改设计数据通路和控制器 (增加中断控制)
- 拓展 CPU 中断功能

任务二: 设计 CPU 中断测试方案并完成测试

原理：

2.1 中断基本概念

中断是指程序执行过程中，当发生某个事件时，中止 CPU 上 现程序的运行，引出处理该事件的程序执行的过程，此过程都需要打断处理器正常的工作。

- 中断源：引起中断的事件称为中断源；
- 中断请求：中断源向 CPU 提出处理的请求；
- 断点：发生中断时被打断程序的暂停点；
- 中断响应：CPU 暂停现程序而转为响应中断请求的过程；
- 中断处理程序：处理中断源的程序；
- 中断处理：CPU 执行有关的中断处理程序；
- 中断返回：返回断点的过程；

当 CPU 收到中断或者异常的信号时，它会暂停执行当前的程序或任务，通过一定的机制跳转到负责处理这个信号的相关处理程序中，在完成对这个信号的处理后再跳回到刚才被打断的程序或任务中。

- 保护 CPU 现场 --> • 处理发生的中断事件 --> • 恢复正常操作

2.2 RISC-V 中断处理寄存器

RISC-V 架构主要分为 3 个工作模式，不同模式均可产生中断：

- 机器模式(Machine Mode) —— 本实验目标
- 用户模式(User Mode)
- 监督模式(Supervisor Mode)

中断处理时涉及多个新寄存器，在此做简单介绍：

(1) 异常入口基址寄存器 - mtvec:

处理器进入异常后，跳入的 PC 地址由 mtvec 指定，本实验采取固定向量。一般，MODE=1 时进行中断的跳转，且跳转地址为 $BASE + 4 * CAUSE$ (CAUSE 为中断对应的编号)。

31	21	0
BASE		MODE

向量地址	ARM异常名称	ARM系统工作模式	本实验定义
0x0000000	复位	超级用户Svc	复位
0x0000004	未定义指令终止	未定义指令终止Und	非法指令异常
0x0000008	软中断（SWI）	超级用户Svc	ECALL
0x000000c	Prefetch abort	指令预取终止Abt	Int外部中断（硬件）
0x0000010	Data abort	数据访问终止Abt	Reserved自定义
0x0000014	Reserved	Reserved	Reserved自定义
0x0000018	IRQ	外部中断模式IRQ	Reserved自定义
0x000001C	FIQ	快速中断模式FIQ	Reserved自定义

【ARM 中断处理跳转地址向量表】

（2）异常原因寄存器 – mcause：

处理器进入异常后，异常的引发原因由 mcause 寄存器指定。

31			30			0		
interrupt			Exception code					

INT	EC	Description
0	0	Instruction address misaligned
0	1	Instruction access fault
0	2	Illegal instruction
0	3	Breakpoint
0	4	Load address misaligned
0	5	Load access fault
0	6	Store/AMO address misaligned

INT	EC	Description
0	7	Store/AMO access fault
0	10..8	Not supported
0	11	Ecall from M-mode
0	>=12	Reserved
1	2..0	Reserved
1	3	Machine Software Interrupt
1	6..4	Reserved
1	7	Machine Timer Interrupt
1	10..8	Reserved
1	11	Machine External Interrupt
1	>=12	Reserved

（3）*异常 PC 寄存器 – mepc：

处理器进入异常后，异常的返回地址由 mepc 寄存器保存。

在进入异常时，硬件将自动更新 mepc 寄存器的值，为当前遇到异常的指令 PC 的下一条指令。Mepc 寄存器作为异常的返回地址，在异常结束后，能够使用它保存的 PC 值回到之前被停止执行的程序点。

（4）异常值寄存器 – mtval：

处理器进入异常后，异常的存储器访问地址或指令编码由 mtval 寄存器保存。

（5）异常状态寄存器 – mstatus：

处理器进入异常后，异常的各种状态由 mstatus 寄存器指示。

2.3 RISC-V 中断处理相关指令

(1) 异常返回 – mret : $PC \leq mepc$

0011000	00010	00000	000	00000	1110011	R mret
---------	-------	-------	-----	-------	---------	--------

当异常程序处理完成后，最终要从异常服务程序中退出，并返回主程序。在机器模式下退出异常时候，软件须使用 MRET，规定：

- 停止执行当前程序流，转而从 csr 寄存器 mepc 定义的 pc 地址开始执行 •

执行 mret 指令不仅会让处理器跳转到上述的 pc 地址开始执行，还会让硬件同时 更新 csr 寄存器机器模式状态寄存器 mstatus。

(2) 环境调用 – ecall : $mepc = ecall$

000000000000	000	00000	000	00000	1110011	I ecall
--------------	-----	-------	-----	-------	---------	---------

软中断，系统命令中断转而执行规定指令。

(3) 断点 – ebreak : $mepc = ebreak$

000000000001		00000	000	00000	1110011	I ebreak
--------------	--	-------	-----	-------	---------	----------

2.3 RISC-V 中断处理设计

参考 ARM 中断向量，实现非法指令异常和外部中断以及 ECALL。

- 1.外部中断 (Int) 触发中断或非法指令 (illegal) 触发异常或 ecall 系统调用
- 2.mtvec 寄存器定义的 PC 值分别 Int 为 0x0c; ecall 为 0x08; illegal 为 0x04
- 3.mepc 寄存器值更新为下一条指令的 PC 值
- 4.执行异常服务程序 (向量跳转后的程序)
- 5.执行 mret 指令，返回 mepc 保存的 PC 处继续程序流

三、实验过程和数据记录

3.1 设计实现 Datapath:

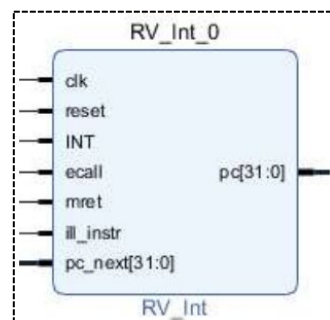
(1) Vivado 新建文件

- 利用 Vivado 新建文件，命名 OExp04-Datapath_int;

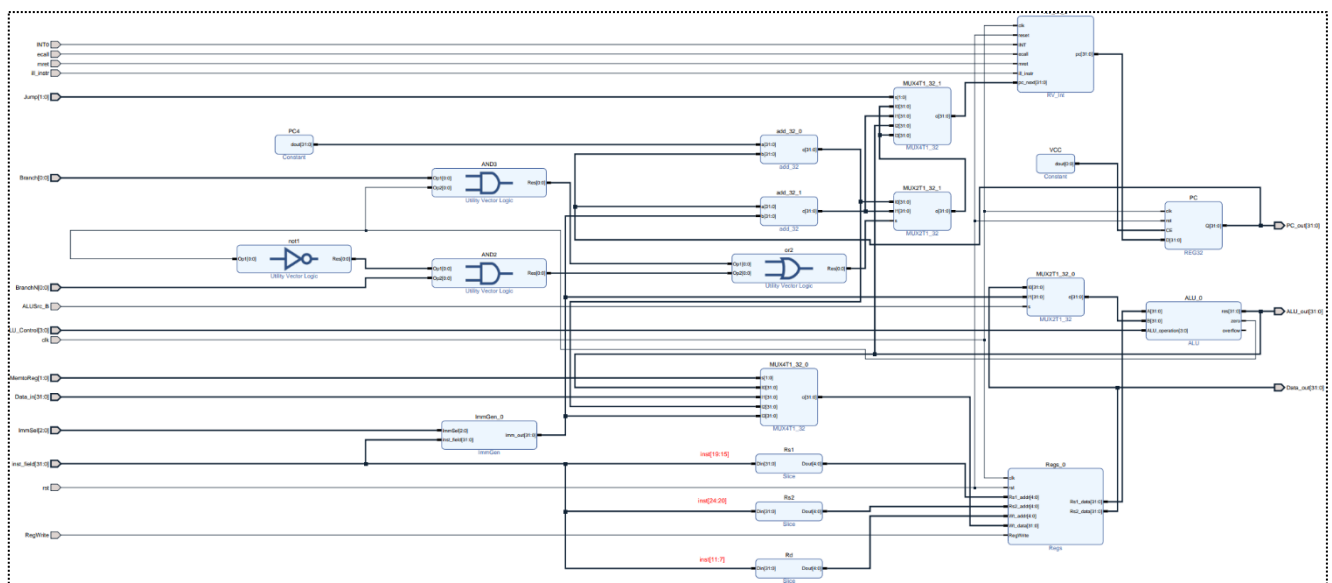
本实验仍然需要用到 Lab0 设计的 MUX 多路选择器、若干基本运算模块 (and、or、xor、nor、srl 等)，以及 Lab1 设计的 Regs。以及 Lab4-03 中修改后的 ALU、ImmGen_more 两个模块 (具体代码见实验原理)。

同时，中断的加入意味着 PC 模块需要增加：

- mtvec 寄存器型变量 (非必需)，中断和异常触发 PC 转向中断地址
 - ◆ 相当于硬件触发 Jal，用 mret 返回
- mepc 寄存器，中断和异常返回 PC 的地址
- 增加控制信号 INT (不要重复响应)、mret、ecall、ill_instr



(2) 设计 Datapath_more:



首先需要完成 RV_int 模块的设计:

```
`timescale 1ns / 1ps

module RV_int(
    input wire clk ,
    input wire reset ,
    input wire INT,
    input wire ecall ,
    input wire mret,
    input wire ill_instr,
    input wire [31:0] pc_next,
    output reg [31:0] pc,
    output reg [31:0] MEPC
);
    always @(posedge clk or posedge reset) begin
        if (reset) begin
            MEPC <= 32'h00000000; // 复位时清除 MEPC
        end
        else if (INT | ecall | ill_instr) begin
            MEPC <= pc_next; // 中断发生时, 保存下一条指令的地址
        end
    end

    always @(*) begin
        if (reset) begin
            pc = 32'h00000000;
        end
        else if (INT) begin
            pc = 32'h0000000c; // 跳转到中断处理入口
        end
        else if (ecall) begin
            pc = 32'h00000008; // 跳转到 ecall 处理入口
        end
        else if (ill_instr) begin
            pc = 32'h00000004; // 跳转到非法指令处理入口
        end
        else if (mret) begin
            pc = MEPC; // 执行 mret 时恢复到 MEPC 保存的下一条指令
        end
        else begin // 如果没有异常, 正常更新 MEPC
            pc = pc_next;
        end
    end
end
```

逻辑原理图已给出，相较于 Datapath_more 的区别只在添加了 RV_int 模块来进行中断处理，代码实现如下：

```
`timescale 1ns / 1ps

module Datapath_int(
    input wire clk,
    input wire rst,
    input wire[31:0] inst_field,
    input wire[31:0] Data_in,
    input wire[3:0] ALU_operation,
    input wire[2:0] ImmSel,
    input wire[1:0] MemtoReg,
    input wire ALUSrc_B,
    input wire [1:0] Jump,
    input wire Branch, // beq
    input wire BranchN, // bne
    input wire RegWrite,

    input wire INT0,
    input wire ecall,
    input wire mret,
    input wire ill_instr,

    output wire[31:0] PC_out,
    output wire[31:0] Data_out,
    output wire[31:0] ALU_out,
    output wire[1023:0] all_regs_data,
    output wire[31:0] mepc
);

    wire[31:0] Imm_out;
    wire[31:0] PC_inc;
    wire[31:0] PC_imm;
    wire ALU_zero;
    wire[31:0] PC_1;
    wire[31:0] Wt_data;
    wire[31:0] PC_new;
    wire[31:0] Rs1_data;
    wire[31:0] ALU_B;
    wire[31:0] PC_int;
```

```

RV_int RV_int(
    .clk(clk),
    .reset(rst),
    .INT(INT0),
    .ecall(ecall),
    .mret(mret),
    .ill_instr(ill_instr),
    .pc_next(PC_new),
    .pc(PC_int),
    .MEPC(mepc));
ImmGen_more ImmGen(
    .ImmSel(ImmSel),
    .inst_field(inst_field),
    .Imm_out(Imm_out));
add32 add_32_0(
    .a(32'h4),
    .b(PC_out),
    .c(PC_inc));
add32 add_32_1(
    .a(PC_out),
    .b(Imm_out),
    .c(PC_imm));
MUX2T1_32 MUX2T1_32_1(
    .I0(PC_inc),
    .I1(PC_imm),
    .s((Branch & ALU_zero) | (BranchN & ~ALU_zero)),
    .o(PC_1));
MUX4T1_32 MUX4T1_32_0(
    .s(MemtoReg),
    .I0(ALU_out),
    .I1(Data_in),
    .I2(PC_inc),
    .I3(Imm_out),
    .o(Wt_data));
MUX4T1_32 MUX4T1_32_1(
    .I0(PC_1),
    .I1(PC_imm),
    .I2(ALU_out),
    .I3(PC_1),
    .s(Jump),
    .o(PC_new));
MUX2T1_32 MUX2T1_32_0(
    .I0(Data_out),
    .I1(Imm_out),

```

```

        .s(ALUSrc_B),
        .o(ALU_B));
    Regs Regs(
        .clk(clk),
        .rst(rst),
        .RegWrite(RegWrite),
        .Rs1_addr(inst_field[19:15]),
        .Rs2_addr(inst_field[24:20]),
        .Wt_addr(inst_field[11:7]),
        .Wt_data(Wt_data),
        .Rs1_data(Rs1_data),
        .Rs2_data(Data_out),
        .all_regs_data(all_regs_data));
    ALU ALU(
        .A(Rs1_data),
        .B(ALU_B),
        .ALU_operation(ALU_operation),
        .res(ALU_out),
        .zero(ALU_zero));
    REG32 PC(
        .clk(clk),
        .rst(rst),
        .CE(1'b1),
        .D(PC_int),
        .Q(PC_out));

endmodule

```

(3) Lab04-4 Datapath_int 工程结构:

- U2 : Datapath_int (Datapath_int.v) (11)
 - RV_int : RV_int (RV_int.v)
 - ImmGen : ImmGen_more (ImmGen_more.v)
 - add_32_0 : add32 (add32.v)
 - add_32_1 : add32 (add32.v)
 - MUX2T1_32_1 : MUX2T1_32 (MUX2T1_32.v)
 - MUX4T1_32_0 : MUX4T1_32 (MUX4T1_32.v)
 - MUX4T1_32_1 : MUX4T1_32 (MUX4T1_32.v)
 - MUX2T1_32_0 : MUX2T1_32 (MUX2T1_32.v)
 - Regs : Regs (Regs.v)
 - ALU : ALU (ALU.v)
 - PC : REG32 (REG32.v)

3.2 设计实现 SCPU_Ctrl_int:

(1) Vivado 新建文件

- 利用 Vivado 新建文件，命名 OExp04-SCPU_ctrl_int;

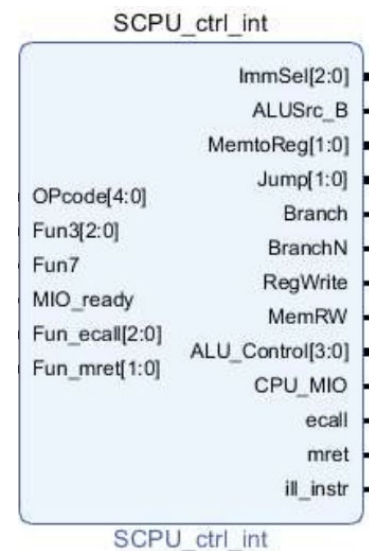
主要实现两个阶段的译码：根据指令 inst_in 对主要控制信号进行赋值分配；根据 ALU_op、Funct3/7 对 ALU 中的 ALU_Control 进行赋值。

本实验额外需要的译码是在 default 部分为 illegal 指令给予中断，并为新添加的 ecall、mret 指令（Opcode = 11100）译码。

(2) 描述设计 SCPU_ctrl_more:

- 逻辑接口:

```
module SCPU_ctrl_int(  
    input [4:0] Opcode, //Opcode  
    input [2:0] Fun3, //Function  
    input Fun7, //Function  
    input MIO_ready, //CPU Wait  
    input [2:0] Fun_ecall ,  
    input [1:0] Fun_mret ,  
  
    output reg [2:0] ImmSel, //立即数选择控制  
    output reg ALUSrc_B, //源操作数 2 选择  
    output reg [1:0] MemtoReg, //写回数据选择控制  
    output reg [1:0] Jump, //jal  
    output reg Branch, //beq  
    output reg BranchN,  
    output reg RegWrite, //寄存器写使能  
    output reg MemRW, //存储器读写使能  
    output reg [3:0] ALU_Control, //alu 控制  
    output reg CPU_MIO ,  
  
    output reg ecall ,  
    output reg mret ,  
    output reg ill_instr  
);  
  
reg [1:0] ALUOp;  
wire [3:0] Fun;  
assign Fun = {Fun3, Fun7};
```



- 主控制器设计:

```

always @(*) begin
    case(OPcode)
5'b01100: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,BranchN,Jump,ALUop,ImmSel} =
14'b0_00_1_0_0_0_00_10_000; ill_instr = 1'b0; ecall = 1'b0; mret =
1'b0;end //ALU
5'b00000: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,BranchN,Jump,ALUop,ImmSel} =
14'b1_01_1_0_0_0_00_00_001; ill_instr = 1'b0; ecall = 1'b0; mret =
1'b0;end //load
5'b01000: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,BranchN,Jump,ALUop,ImmSel} =
14'b1_00_0_1_0_0_00_00_010; ill_instr = 1'b0; ecall = 1'b0; mret =
1'b0;end //store
5'b11000: begin
    case(Fun3)
3'b000: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,BranchN,Jump,ALUop,ImmSel} =
14'b0_00_0_0_1_0_00_01_011; ill_instr = 1'b0; ecall = 1'b0; mret =
1'b0; end // beq
3'b001: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,BranchN,Jump,ALUop,ImmSel} =
14'b0_00_0_0_0_1_00_01_011; ill_instr = 1'b0; ecall = 1'b0; mret =
1'b0; end // bne
default : begin ill_instr = 1'b1; ecall = 1'b0; mret = 1'b0; end
    endcase
    end
5'b11011: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,BranchN,Jump,ALUop,ImmSel} =
14'b1_10_1_0_0_0_01_00_100; ill_instr = 1'b0; ecall = 1'b0; mret =
1'b0;end //jump
5'b00100: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,BranchN,Jump,ALUop,ImmSel} =
14'b1_00_1_0_0_0_00_11_001; ill_instr = 1'b0; ecall = 1'b0; mret =
1'b0;end //ALU(addi;;;)
5'b11001: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,BranchN,Jump,ALUop,ImmSel} =
14'b1_10_1_0_0_0_10_00_001; ill_instr = 1'b0; ecall = 1'b0; mret =
1'b0;end //jarl
5'b01101: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,BranchN,Jump,ALUop,ImmSel} =
14'b0_11_1_0_0_0_00_00_000; ill_instr = 1'b0; ecall = 1'b0; mret =
1'b0;end //lui

```

```

5'b11100: begin
    {ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,BranchN,Jump,ALUop,ImmSel}
    = 14'h0;
    if(Fun_mret == 2'b00 && Fun_ecall == 3'b000) begin
        ecall = 1'b1;
        mret = 1'b0;
        ill_instr = 1'b0;
    end
    else if(Fun_mret == 2'b11 && Fun_ecall == 3'b010) begin
        mret = 1'b1;
        ecall = 1'b0;
        ill_instr = 1'b0;
    end
    else begin
        ill_instr = 1'b1;
        mret = 1'b0;
        ecall = 1'b0;
    end
    end
    default: begin ill_instr = 1'b1; ecall = 1'b0; mret = 1'b0; end
    endcase
end

```

- ALU 译码控制设计:

```

always @(*) begin
    case(ALUop)
        2'b00: ALU_Control = 4'b0010 ;
        2'b01: ALU_Control = 4'b0110 ;
        2'b10:
            case(Fun)
                4'b0000: ALU_Control = 4'b0010 ; //add
                4'b0001: ALU_Control = 4'b0110 ; //sub
                4'b1110: ALU_Control = 4'b0000 ; //and
                4'b1100: ALU_Control = 4'b0001 ; //or
                4'b0100: ALU_Control = 4'b0111 ; //slt
                4'b1010: ALU_Control = 4'b1101 ; //srl
                4'b1000: ALU_Control = 4'b1100 ; //xor
                4'b0110: ALU_Control = 4'b1001 ; //sltu
                4'b0010: ALU_Control = 4'b1110 ; //sll
                4'b1011: ALU_Control = 4'b1111 ; //sra
            default: ALU_Control = 4'bx;
            endcase
    endcase
end

```

```

2'b11:
    case(Fun3)
        3'b000: ALU_Control = 4'b0010 ; //addi
        3'b010: ALU_Control = 4'b0111 ; //slti
        3'b100: ALU_Control = 4'b1100 ; //xori
        3'b110: ALU_Control = 4'b0001 ; //ori
        3'b111: ALU_Control = 4'b0000 ; //andi
        3'b101:
            if(Fun7)
                ALU_Control = 4'b1111 ; //srai
            else
                ALU_Control = 4'b1101 ; //srli
        3'b011: ALU_Control = 4'b1001 ; //sltiu
        3'b001: ALU_Control = 4'b1110 ; //slli
        default: ALU_Control = 4'bx ;
    endcase
endcase
end

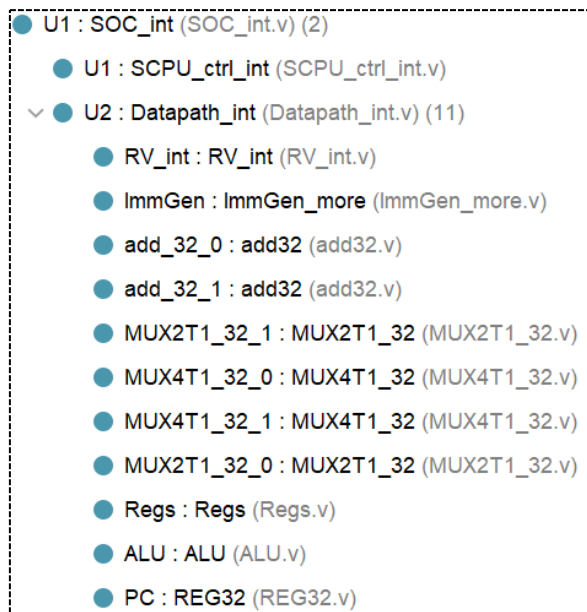
```

3.3 集成 CPU

3.3.1 搭建仿真平台

通过 3.1、3.2, 我们通过逻辑代码实现了 Datapath_int 和 SCPU_ctrl_int 模块, 对 CPU 进行了指令拓展, 我们参考 Lab4-02 的操作也能建立一个 CPU 仿真测试平台, 对相关指令进行检测。

首先对两个模块进行集成组合成 CPU:



【具体代码如下】

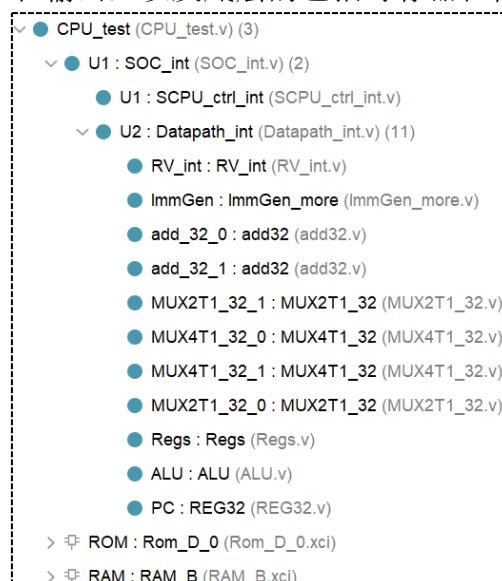
```
`timescale 1ns / 1ps
module SOC_int(
    input clk,
    input rst,
    input [31:0]Data_in,
    input [31:0]inst_in,
    input MIO_ready,
    inout INT0,
    output MemRW,
    output CPU_MIO,
    output [31:0]Data_out,
    output [31:0]PC_out,
    output [31:0]Addr_out,
    output [1023:0] all_regs_data,
    output [31:0]mepc
);
    wire [3:0]ALU_Control;
    wire [2:0]ImmSel;
    wire [1:0]MemtoReg,Jump;
    wire Branch, BranchN, RegWrite, ALUSrc_B;
    wire ecall , mret, ill_instr;
    wire[4:0] opcode;    assign opcode = inst_in[6:2];
    wire[3:0] Fun3;      assign Fun3 = inst_in[14:12];
    wire[3:0] funecall;  assign funecall = inst_in[22:20];
    wire[2:0] funmret;   assign funmret = inst_in[29:28];
    SCPU_ctrl_int U1(
        .OPcode(opcode),
        .Fun3(Fun3),
        .Fun7(inst_in[30]),
        .MIO_ready(MIO_ready),
        .Fun_ecall(funecall),
        .Fun_mret(funmret),
        .ImmSel(ImmSel),
        .ALUSrc_B(ALUSrc_B),
        .MemtoReg(MemtoReg),
        .Jump(Jump),
        .Branch(Branch),
        .BranchN(BranchN),
        .RegWrite(RegWrite),
        .MemRW(MemRW),
        .ALU_Control(ALU_Control),
        .CPU_MIO(CPU_MIO),
        .ecall(ecall),
```

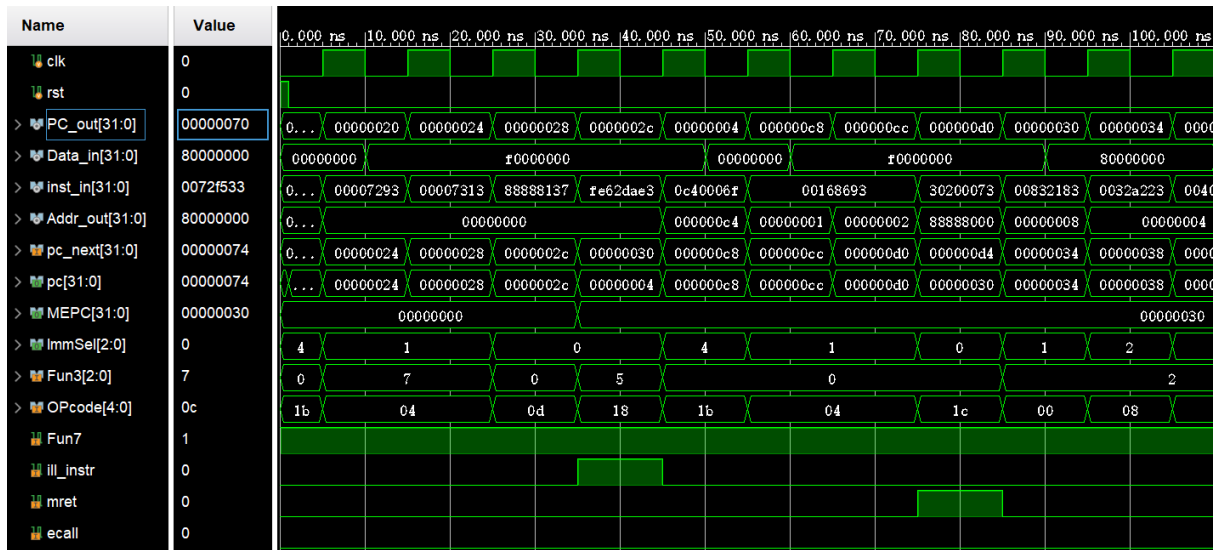
```

        .mret(mret),
        .ill_instr(ill_instr));
Datapath_int U2(
    .ALUSrc_B(ALUSrc_B),
    .ALU_operation(ALU_Control),
    .Branch(Branch),
    .BranchN(BranchN),
    .Data_in(Data_in),
    .ImmSel(ImmSel),
    .Jump(Jump),
    .MemtoReg(MemtoReg),
    .RegWrite(RegWrite),
    .clk(clk),
    .inst_field(inst_in),
    .rst(rst),
    .ALU_out(Addr_out),
    .Data_out(Data_out),
    .PC_out(PC_out),
    .all_regs_data(all_regs_data),
    .INT0(INT0),
    .ecall(ecall),
    .ill_instr(ill_instr),
    .mret(mret),
    .mepc(mepc));
endmodule

```

同样的，利用 CPU（本实验设计）、RAM、ROM 可以搭建如下图的仿真平台，初始化 RAM 的数据（D_mem.coe），初始化 ROM 的指令（I_mem_int.coe），查看 CPU 数据的输入和输出，以及底层的包括寄存器在内的各模块的状态。

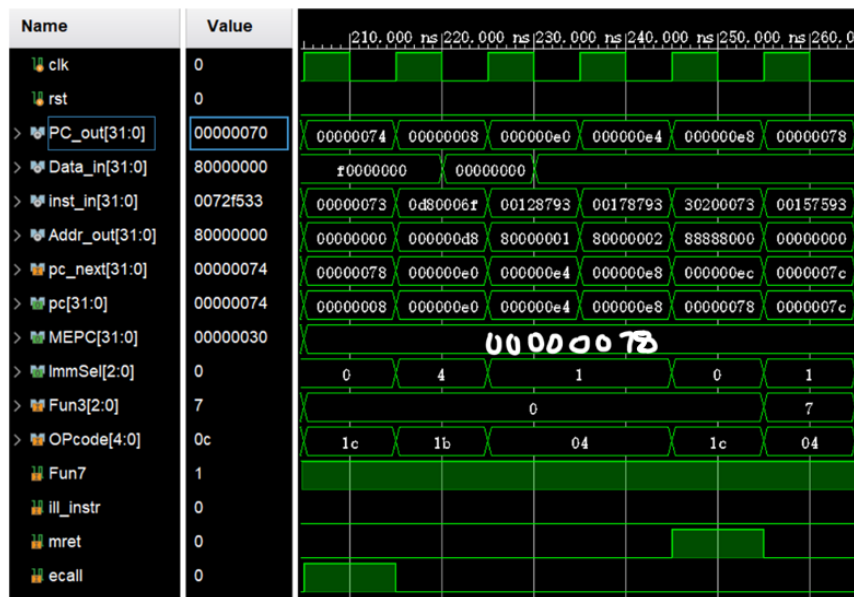




该片段中，PC = 0x2c 时指令为 0xFE62DAE3，对应机器码为 bge x5, x6, -12 不在我们的设计范围中，因此判定为 illegal instruction 需要中断。

下一个时钟周期，可以看到 PC 跳转到 ill_init 处理向量位置 0x04，同时 MEPC 记录下一指令 0x30，并转向中断处理程序 0xc8 进行处理。

处理完毕，进行 mret，返回到正常程序流。



该片段中，PC = 0x74 时指令为 0X00000073，对应机器码为 ecall，因此判定需要中断。

下一个时钟周期，可以看到 PC 跳转到 ecall 处理向量位置 0x08，同时 MEPC 记录下一指令 0x78，并转向中断处理程序 0xe0 进行处理。

处理完毕，进行 mret，返回到正常程序流。

3.3.2 DEMO 物理测试

继承替换 CPU 及子模块后，下载.bit 文件，使用 DEMO 程序进行物理验证。

本实验由于指令种类多样并且指令较为复杂，通过记录 VGA 中的数据变化，设计了测试记录表格。

- ROM 中填充指令（I_mem_int.coe）如下：

```
memory_initialization_radix=16;  
memory_initialization_vector=  
0200006F,0C40006F,0D80006F,0C80006F,00000033,00000033,00000033,00000033,00007293,00007313,  
88888137,FE62DAE3,00832183,0032A223,00402083,01C02383,00338863,555550B7,0070A0B3,FE0098E3,  
007282B3,00230333,00531463,40000033,40530433,405304B3,0080006F,00007033,0072F533,00000073,  
00157593,00B51463,00006033,00A5E5B3,0015E513,00558463,00004033,00A5C633,00164613,00B61463,  
00000013,0012D293,00060463,40000033,00129293,00B28463,00000013,001026B3,00503733,F5DFF06F,  
00168693,00168693,30200073,40C70733,40C70733,30200073,00128793,00178793,30200073
```

- RAM 中储存的数据（D_mem.coe）如下：

```
memory_initialization_radix=16;  
memory_initialization_vector=  
F0000000, 000002AB, 80000000, 0000003F, 00000001, FFF70000,  
0000FFFF, 80000000, 00000000, 11111111, 22222222, 33333333,  
44444444, 55555555, 66666666, 77777777, 88888888, 99999999,  
aaaaaaaa, bbbbbbbb, cccccccc, dddddddd, eeeeeeee, FFFFFFFF,  
557EF7E0, D7BDFBD9, D7BDFBD9, DFCFFCFB, DFCFBFFF, F7F3DFFF,  
FFFFDF3D, FFFF9DB9, FFFFBCFB, DFCFFCFB, DFCFBFFF, D7DB9FFF,  
D7BDFBD9, D7BDFBD9, FFFF07E0, 007E0FFF, 03bdf020, 03def820,  
08002300
```

部分测试 VGA 显示图片见实验结果分析

四、实验结果分析

PC	Machine Code	Code Basic	Status	Result	MEPC
0x0	0x0200006F	jal x0 32		go to 0x20	0x00
0x4	0x0C40006F	jal x0 196	ill_init	go to 0xc8	
0x8	0x0D80006F	jal x0 216	ecall_init	go to 0xe0	
0xc	0x0C80006F	jal x0 200	int_init	go to 0xd4	
0x10 - 0x1c	0x00000033	add x0 x0 x0	no use		
0x20	0x00007293	andi x5 x0 0		x5 = 0x0	
0x24	0x00007313	andi x6 x0 0		x6 = 0x0	
0x28	0x88888137	lui x2 559240		x2 = 0x88888000	
0x2c	0xFE62DAE3	bge x5 x6 -12	illegal	go to 0x04	0x30
0x30	0x00832183	lw x3 8(x6)		x3 = 0x80000000	
0x34	0x0032A223	sw x3 4(x5)		mem[1] = 0x80000000	
0x38	0x00402083	lw x1 4(x0)		x1 = 0x80000000	
0x3c	0x01C02383	lw x7 28(x0)		x7 = 0x80000000	
0x40	0x00338863	beq x7 x3 16		go to 0x50	
0x44	0x555550B7	lui x1 349525	Jump		
0x48	0x0070A0B3	slt x1 x1 x7	Jump		
0x4c	0xFE0098E3	bne x1 x0 -16	Jump		
0x50	0x007282B3	add x5 x5 x7		x5 = 0x80000000	
0x54	0x00230333	add x6 x6 x2		x6 = 0x88888000	
0x58	0x00531463	bne x6 x5 8		go to 0x60	
0x5c	0x40000033	sub x0 x0 x0	Jump		
0x60	0x40530433	sub x8 x6 x5		x8 = 0x08888000	
0x64	0x405304B3	sub x9 x6 x5		x9 = 0x08888000	
0x68	0x0080006F	jal x0 8		go to 0x70	
0x6c	0x00007033	and x0 x0 x0	Jump		
0x70	0x0072F533	and x10 x5 x7		x10 = 0x80000000	
0x74	0x00000073	ecall	ecall	go to 0x08	0x78
0x78	0x00157593	andi x11 x10 1		x11 = 0x0	
0x7c	0x00B51463	bne x10 x11 8		go to 0x84	
0x80	0x00006033	or x0 x0 x0	Jump		
0x84	0x00A5E5B3	or x11 x11 x10		x11 = 0x80000000	
0x88	0x0015E513	ori x10 x11 1		x10 = 0x80000001	
0x8c	0x00558463	beq x11 x5 8		go to 0x94	
0x90	0x00004033	xor x0 x0 x0	Jump		
0x94	0x00A5C633	xor x12 x11 x10		x12 = 0x00000001	
0x98	0x00164613	xori x12 x12 1		x12 = 0x0	
0x9c	0x00B61463	bne x12 x11 8		go to a4	
0xa0	0x00000013	addi x0 x0 0	Jump		
0xa4	0x0012D293	srl x5 x5 1		x5 = 0x40000000	
0xa8	0x00060463	beq x12 x0 8		go to 0xb0	
0xac	0x40000033	sub x0 x0 x0	Jump		
0xb0	0x00129293	slli x5 x5 1		x5 = 0x80000000	
0xb4	0x00B28463	beq x5 x11 8		go to 0xbc	
0xb8	0x00000013	addi x0 x0 0	Jump		
0xbc	0x001026B3	slt x13 x0 x1		x13 = 0x0	
0xc0	0x00503733	sltu x14 x0 x5		x14 = 0x00000001	
0xc4	0xF5DFF06F	jal x0 -164		go to 0x0	

以上表格记录了 I_mem_int.coe 中的正常程序流，并标记了中断处。

0xc8	0x00168693	addi x13 x13 1	ill_init	x13 = 0x00000001	0x30
0xcc	0x00168693	addi x13 x13 1		x13 = 0x00000002	0x30
0xd0	0x30200073	mret	mret	go to PC = MEPC	0x30
0xd4	0x40C70733	sub x14 x14 x12			
0xd8	0x40C70733	sub x14 x14 x12			
0xdc	0x30200073	mret			
0xe0	0x00128793	addi x15 x5 1	ecall	x15 = 0x80000001	0x78
0xe4	0x00178793	addi x15 x15 1		x15 = 0x80000002	0x78
0xe8	0x30200073	mret	mret	go to PC = MEPC	0x78

该表格记录了中断处理程序，其中 MEPC 保持在进入中断时程序的下一条 PC，从而能够在 mret 时重回正常程序流。

(1) Illegal instruction 中断:

```
RV32I Single Cycle CPU
pc: 0000002c   inst: fe62dae3

x0: 00000000   ra: 00000000   sp: 88888000   gp: 00000000   tp: 00000000
t0: 00000000   t1: 00000000   t2: 00000000   s0: 00000000   s1: 00000000
a0: 00000000   a1: 00000000   a2: 00000000   a3: 00000000   a4: 00000000
a5: 00000000   a6: 00000000   a7: 00000000   s2: 00000000   s3: 00000000
s4: 00000000   s5: 00000000   s6: 00000000   s7: 00000000   s8: 00000000
s9: 00000000   s10: 00000000   s11: 00000000   t3: 00000000   t4: 00000000
t5: 00000000   t6: 00000000

rs1: 00   rs1_val: 00000000
rs2: 00   rs2_val: 00000000
rd: 00   reg_i_data: 00000000   reg_wen: 0

is_imm: 0   is_auiopc: 0   is_lui: 0   imm: 00000000
a_val: 00000000   b_val: 00000000   alu_ctrl: 0   cmp_ctrl: 0
alu_res: 00000000   cmp_res: 0

is_branch: 0   is_jal: 0   is_jalr: 0
do_branch: 0   pc_branch: 00000000

mem_wen: 0   mem_ren: 0
dmem_o_data: f0000000   dmem_i_data: 00000000   dmem_addr: 00000000

csr_wen: 0   csr_ind: 000   csr_ctrl: 0   csr_r_data: 00000000
mstatus: 00000000   mcause: 00000000   mepc: 00000000   mtval: 00000000
mtvec: 00000000   mie: 00000000   mip: 00000000
```

【Instruction = 0xFE62DAE3 = bge x5 , x6 , -12, 记录 MEPC=0x30】

```
RV32I Single Cycle CPU
pc: 00000004   inst: 0c40006f

x0: 00000000   ra: 00000000   sp: 88888000   gp: 00000000   tp: 00000000
t0: 00000000   t1: 00000000   t2: 00000000   s0: 00000000   s1: 00000000
a0: 00000000   a1: 00000000   a2: 00000000   a3: 00000000   a4: 00000000
a5: 00000000   a6: 00000000   a7: 00000000   s2: 00000000   s3: 00000000
s4: 00000000   s5: 00000000   s6: 00000000   s7: 00000000   s8: 00000000
s9: 00000000   s10: 00000000   s11: 00000000   t3: 00000000   t4: 00000000
t5: 00000000   t6: 00000000

rs1: 00   rs1_val: 00000000
rs2: 00   rs2_val: 00000000
rd: 00   reg_i_data: 00000000   reg_wen: 0

is_imm: 0   is_auiopc: 0   is_lui: 0   imm: 00000000
a_val: 00000000   b_val: 00000000   alu_ctrl: 0   cmp_ctrl: 0
alu_res: 000000c4   cmp_res: 0

is_branch: 0   is_jal: 0   is_jalr: 0
do_branch: 0   pc_branch: 00000000

mem_wen: 0   mem_ren: 0
dmem_o_data: 00000000   dmem_i_data: 00000000   dmem_addr: 000000c4

csr_wen: 0   csr_ind: 000   csr_ctrl: 0   csr_r_data: 00000000
mstatus: 00000000   mcause: 00000000   mepc: 00000000   mtval: 00000000
mtvec: 00000000   mie: 00000000   mip: 00000000
```

【Illegal 中断发生, 跳转至 0x4, 准备跳入中断处理程序】

```
RV32I Single Cycle CPU
pc: 000000d0   inst: 30200073

x0: 00000000   ra: 00000000   sp: 88888000   gp: 00000000   tp: 00000000
t0: 00000000   t1: 00000000   t2: 00000000   s0: 00000000   s1: 00000000
a0: 00000000   a1: 00000000   a2: 00000000   a3: 00000002   a4: 00000000
a5: 00000000   a6: 00000000   a7: 00000000   s2: 00000000   s3: 00000000
s4: 00000000   s5: 00000000   s6: 00000000   s7: 00000000   s8: 00000000
s9: 00000000   s10: 00000000   s11: 00000000   t3: 00000000   t4: 00000000
t5: 00000000   t6: 00000000

rs1: 00   rs1_val: 00000000
rs2: 00   rs2_val: 00000000
rd: 00   reg_i_data: 00000000   reg_wen: 0

is_imm: 0   is_auiopc: 0   is_lui: 0   imm: 00000000
a_val: 00000000   b_val: 00000000   alu_ctrl: 0   cmp_ctrl: 0
alu_res: 88888000   cmp_res: 0

is_branch: 0   is_jal: 0   is_jalr: 0
do_branch: 0   pc_branch: 00000000

mem_wen: 0   mem_ren: 0
dmem_o_data: f0000000   dmem_i_data: 00000000   dmem_addr: 88888000

csr_wen: 0   csr_ind: 000   csr_ctrl: 0   csr_r_data: 00000000
mstatus: 00000000   mcause: 00000000   mepc: 00000000   mtval: 00000000
mtvec: 00000000   mie: 00000000   mip: 00000000
```

【0xc8~0xd0 执行成功, x13=0x2, 随后跳回 0x30】

(2) ECALL 中断:

```
RV32I Single Cycle CPU
pc: 00000074 inst: 00000073
x0: 00000000 ra: 80000000 sp: 88888000 gp: 80000000 tp: 00000000
t0: 80000000 t1: 88888000 t2: 80000000 s0: 88888000 s1: 88888000
a0: 80000000 a1: 00000000 a2: 00000000 a3: 00000002 a4: 00000000
a5: 00000000 a6: 00000000 a7: 00000000 s2: 00000000 s3: 00000000
s4: 00000000 s5: 00000000 s6: 00000000 s7: 00000000 s8: 00000000
s9: 00000000 s10: 00000000 s11: 00000000 t3: 00000000 t4: 00000000
t5: 00000000 t6: 00000000

rs1: 00 rs1_val: 00000000
rs2: 00 rs2_val: 00000000
rd: 00 reg_i_data: 00000000 reg_wen: 0

is_imm: 0 is_auiopc: 0 is_lui: 0 imm: 00000000
a_val: 00000000 b_val: 00000000 alu_ctrl: 0 cmp_ctrl: 0
alu_res: 00000000 cmp_res: 0

is_branch: 0 is_jal: 0 is_jalr: 0
do_branch: 0 pc_branch: 00000000

mem_wen: 0 mem_ren: 0
dmem_o_data: f0000000 dmem_i_data: 00000000 dmem_addr: 00000000

csr_wen: 0 csr_ind: 000 csr_ctrl: 0 csr_r_data: 00000000
mstatus: 00000000 mcause: 00000000 mepc: 00000000 mtval: 00000000
mtvec: 00000000 mie: 00000000 mip: 00000000
```

【Instruction = 0x00000073 = ecall, 记录 MEPC=0x78】

```
RV32I Single Cycle CPU
pc: 00000008 inst: 0d80006f
x0: 00000000 ra: 80000000 sp: 88888000 gp: 80000000 tp: 00000000
t0: 80000000 t1: 88888000 t2: 80000000 s0: 88888000 s1: 88888000
a0: 80000000 a1: 00000000 a2: 00000000 a3: 00000002 a4: 00000000
a5: 00000000 a6: 00000000 a7: 00000000 s2: 00000000 s3: 00000000
s4: 00000000 s5: 00000000 s6: 00000000 s7: 00000000 s8: 00000000
s9: 00000000 s10: 00000000 s11: 00000000 t3: 00000000 t4: 00000000
t5: 00000000 t6: 00000000

rs1: 00 rs1_val: 00000000
rs2: 00 rs2_val: 00000000
rd: 00 reg_i_data: 00000000 reg_wen: 0

is_imm: 0 is_auiopc: 0 is_lui: 0 imm: 00000000
a_val: 00000000 b_val: 00000000 alu_ctrl: 0 cmp_ctrl: 0
alu_res: 000000d8 cmp_res: 0

is_branch: 0 is_jal: 0 is_jalr: 0
do_branch: 0 pc_branch: 00000000

mem_wen: 0 mem_ren: 0
dmem_o_data: 00000000 dmem_i_data: 00000000 dmem_addr: 000000d8

csr_wen: 0 csr_ind: 000 csr_ctrl: 0 csr_r_data: 00000000
mstatus: 00000000 mcause: 00000000 mepc: 00000000 mtval: 00000000
mtvec: 00000000 mie: 00000000 mip: 00000000
```

【Ecall 中断发生, 跳转至 0x8, 准备跳入中断处理程序】

```
RV32I Single Cycle CPU
pc: 000000e8 inst: 30200073
x0: 00000000 ra: 80000000 sp: 88888000 gp: 80000000 tp: 00000000
t0: 80000000 t1: 88888000 t2: 80000000 s0: 88888000 s1: 88888000
a0: 80000000 a1: 00000000 a2: 00000000 a3: 00000002 a4: 00000000
a5: 80000002 a6: 00000000 a7: 00000000 s2: 00000000 s3: 00000000
s4: 00000000 s5: 00000000 s6: 00000000 s7: 00000000 s8: 00000000
s9: 00000000 s10: 00000000 s11: 00000000 t3: 00000000 t4: 00000000
t5: 00000000 t6: 00000000

rs1: 00 rs1_val: 00000000
rs2: 00 rs2_val: 00000000
rd: 00 reg_i_data: 00000000 reg_wen: 0

is_imm: 0 is_auiopc: 0 is_lui: 0 imm: 00000000
a_val: 00000000 b_val: 00000000 alu_ctrl: 0 cmp_ctrl: 0
alu_res: 88888000 cmp_res: 0

is_branch: 0 is_jal: 0 is_jalr: 0
do_branch: 0 pc_branch: 00000000

mem_wen: 0 mem_ren: 0
dmem_o_data: f0000000 dmem_i_data: 00000000 dmem_addr: 88888000

csr_wen: 0 csr_ind: 000 csr_ctrl: 0 csr_r_data: 00000000
mstatus: 00000000 mcause: 00000000 mepc: 00000000 mtval: 00000000
mtvec: 00000000 mie: 00000000 mip: 00000000
```

【0xe0~0xe8 执行成功, x15=0x2, 随后跳回 0x78】

(3) INT 外部中断:

在运行过程中, 将 SW[15]调至 1, 并进行一步单步时钟, INT 外部中断发生, 跳转至 0x8, 准备跳入中断处理程序。

```
RV32I Single Cycle CPU
pc: 0000000c   inst: 0c80006f
x0: 00000000   ra: 80000000   sp: 88888000   gp: 80000000   tp: 00000000
t0: 00000000   t1: 88888000   t2: 80000000   s0: 08888000   s1: 08888000
a0: 00000000   a1: 00000000   a2: 00000000   a3: 00000002   a4: 00000000
s4: 00000000   s5: 00000000   s6: 00000000   s7: 00000000   s8: 00000000
s9: 00000000   s10: 00000000   s11: 00000000   t3: 00000000   t4: 00000000
t5: 00000000   t6: 00000000

rs1: 00   rs1_val: 00000000
rs2: 00   rs2_val: 00000000
rd: 00   reg_i_data: 00000000   reg_wen: 0

is_imm: 0   is_auiopc: 0   is_lui: 0   imm: 00000000
a_val: 00000000   b_val: 00000000   alu_ctrl: 0   cmp_ctrl: 0
alu_res: 00000008   cmp_res: 0

is_branch: 0   is_jal: 0   is_jalr: 0
do_branch: 0   pc_branch: 00000000

mem_wen: 0   mem_ren: 0
dmem_o_data: 00000000   dmem_i_data: 08888000   dmem_addr: 00000008

csr_wen: 0   csr_ind: 000   csr_ctrl: 0   csr_r_data: 00000000
mstatus: 00000000   mcause: 00000000   mepc: 00000000   mtval: 00000000
mtvec: 00000000   mie: 00000000   mip: 00000000
```

随后 0xd4~0xdc 执行成功, 随后跳回原程序流。

```
RV32I Single Cycle CPU
pc: 000000d8   inst: 40c70733
x0: 00000000   ra: 80000000   sp: 88888000   gp: 80000000   tp: 00000000
t0: 00000000   t1: 00000000   t2: 80000000   s0: 08888000   s1: 08888000
a0: 00000001   a1: 00000000   a2: 00000000   a3: 00000000   a4: 00000001
s4: 00000000   s5: 00000000   s6: 00000000   s7: 00000000   s8: 00000000
s9: 00000000   s10: 00000000   s11: 00000000   t3: 00000000   t4: 00000000
t5: 00000000   t6: 00000000

rs1: 00   rs1_val: 00000000
rs2: 00   rs2_val: 00000000
rd: 00   reg_i_data: 00000000   reg_wen: 0

is_imm: 0   is_auiopc: 0   is_lui: 0   imm: 00000000
a_val: 00000000   b_val: 00000000   alu_ctrl: 0   cmp_ctrl: 0
alu_res: 00000001   cmp_res: 0

is_branch: 0   is_jal: 0   is_jalr: 0
do_branch: 0   pc_branch: 00000000

mem_wen: 0   mem_ren: 0
dmem_o_data: f0000000   dmem_i_data: 00000000   dmem_addr: 00000001

csr_wen: 0   csr_ind: 000   csr_ctrl: 0   csr_r_data: 00000000
mstatus: 00000000   mcause: 00000000   mepc: 00000000   mtval: 00000000
mtvec: 00000000   mie: 00000000   mip: 00000000
```

(4) 最终寄存器堆数值:

```
RV32I Single Cycle CPU
pc: 000000c4   inst: f5dff06f
x0: 00000000   ra: 80000000   sp: 88888000   gp: 80000000   tp: 00000000
t0: 80000000   t1: 88888000   t2: 80000000   s0: 08888000   s1: 08888000
a0: 80000001   a1: 80000000   a2: 00000000   a3: 00000000   a4: 00000001
s4: 00000000   s5: 00000000   s6: 00000000   s7: 00000000   s8: 00000000
s9: 00000000   s10: 00000000   s11: 00000000   t3: 00000000   t4: 00000000
t5: 00000000   t6: 00000000

rs1: 00   rs1_val: 00000000
rs2: 00   rs2_val: 00000000
rd: 00   reg_i_data: 00000000   reg_wen: 0

is_imm: 0   is_auiopc: 0   is_lui: 0   imm: 00000000
a_val: 00000000   b_val: 00000000   alu_ctrl: 0   cmp_ctrl: 0
alu_res: ffffffff   cmp_res: 0

is_branch: 0   is_jal: 0   is_jalr: 0
do_branch: 0   pc_branch: 00000000

mem_wen: 0   mem_ren: 0
dmem_o_data: f0000000   dmem_i_data: 00000000   dmem_addr: ffffffff

csr_wen: 0   csr_ind: 000   csr_ctrl: 0   csr_r_data: 00000000
mstatus: 00000000   mcause: 00000000   mepc: 00000000   mtval: 00000000
mtvec: 00000000   mie: 00000000   mip: 00000000
```

最后呈上无 INT 外部中断, 正确跑完 I_mem_int.coe 的结果。

五、讨论与心得

思考题 1：设计 SCPU_ctrl_int 时，所增加的特权指令与 RV32I 基本指令译码有何区别？

新增加的 特权指令，不依赖于基本指令中所用的控制信号 ALUSrc_B, MemtoReg, RegWrite, MemRW, Branch, BranchN, Jump, ALUop, ImmSel 等。

而主要特权指令决定于 32 位指令中的 Fun_mret 和 Fun_ecall，以及未在基本指令译码范围出现导致 default 中 ill_instr 的变化。

所以说目前新增加的 特权指令只需要 3 个控制信号就能正确执行，相对来说译码更加简洁明了（虽然可能是加的比较少）。

思考题 2：指令集规定的中断寄存器 mepc、mtvec 均为时钟控制寄存器，本实验在设计 RV_int 时若也都设计为寄存器，能实现 PC 的正常跳转吗？

不能正常跳转，如果 RV_int 也设计成寄存器，可以考虑一种情况，在同一个时钟上升沿和下降沿之间发生一次中断，那么在下次时钟上升沿前 RV_int 中的数值就不能改变，也就无法探测到异常的发生。

这就达不到中断处理的要求，PC 跳转不正常。

单周期 CPU 中断实验结束（太折磨了。。），那接下来就要做流水线 CPU 了。在做单周期的一系列实验中，主要时间和工作集中在 Datapath、SCPU_ctrl 的调整、debug 上，做完以后基本上对每个指令的通路以及控制信号都比较熟悉了。

希望后续流水线 CPU 能够友善一点。

【中间存在一个未解决的问题，如果 ROM 中存在 sw 指令，很大概率会导致内存出错，导致后续 lw 数据出现问题？但是多烧几次以后也有概率获得正确结果，真的太折磨了。】